

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Кафедра «Програмна інженерія та
інтелектуальні технології управління»

ЗВІТ
з лабораторної роботи № 1
з навчальної дисципліни
«WEB-ДОДАТКИ НА ОСНОВІ JAVA»

ВИКОНАВ

Студентка групи КН-222а

Інеса РЕПЕШКО

ПЕРЕВІРИВ

Доцент кафедри ПІІТУ

кандидат технічних наук

Владислав КОВТУН

Харків 2024

ДОСЛІДЖЕННЯ СТЕКУ ТЕХНОЛОГІЙ JAVA ДЛЯ СТВОРЕННЯ ТА РОЗГОРТАННЯ ВЕБЗАСТОСУНКІВ ІЗ ЗАСТОСУВАННЯМ СЕРВЛЕТІВ ТА JSP

Мета

- 1 Закріпити знання про розгортання та налаштування програмного забезпечення необхідного для розробки вебзастосунків із застосуванням стеку технологій Java EE.
- 2 Набути знань про структуру вебзастосунків на Java та порядок їх розробки.
- 3 Вивчити порядок створення проєктів вебзастосунків у обраному інтегрованому середовищі розробки.
- 4 Набути навичок створення сервлетів та розробки вебсторінок із застосуванням сервлетів.
- 5 Набути навичок реалізації переходів між сторінками із застосуванням методів forward та sendRedirect().
- 6 Набути навичок створення Java Server Pages (JSP).
- 7 Набути навичок реалізації обміну даними між сервлетами та JSP.

Завдання роботи

- 1 Встановити програмне забезпечення для розробки вебзастосунків на Java та розробити вебзастосунок.
 - 1.1 Встановити інтегроване середовище розробки для створення вебзастосунків.
 - 1.2 Встановити вебсервер (web server) або сервер застосунків (application server) для розгортання вебзастосунків на Java.
 - 1.3 Створити та налаштувати проєкт у встановленому інтегрованому середовищі розробки.

1.4 Створити вебзастосунок у встановленому інтегрованому середовищі розробки та розгорнути його на встановленому сервері. Вебзастосунок має відображати на екрані наступні дані про студента:

- текстовий напис “Hello, Web World!!!”;
- прізвище та ім’я студента;
- номер групи, в якій навчається студент;
- приналежність студента до департаменту;
- рік виконання роботи.

2 Розробити вебзастосунок, який дозволяє відобразити на веб-сторінці перелік співробітників деякої ІТ-компанії.

3 Модифікувати проєкт, створений після виконання завдання 2, шляхом додавання функціоналу переходу на готову сторінку із сервлету, використовуючи команди `forward()` та `sendRedirect()`.

4 Розробити вебзастосунок, який містить наступний функціонал:

- відображення стартової сторінки із полями для вводу логіну та паролю, після вводу правильної пари відкриває головну сторінку застосунку;
- відображення на головній сторінці меню, що дозволяє перейти до сторінки JSP із списком співробітників у вигляді таблиці без застосування можливостей Expression Language (EL) та Java Standard Tag Library (JSTL).

Хід роботи

1 Встановлення програмного забезпечення для розробки вебзастосунків на Java та розробка вебзастосунку

1.1 Встановлення інтегрованого середовища розробки для створення вебзастосунків

Найбільш популярними IDE для розробки вебзастосунків є IntelliJ Idea Ultimate Edition та Eclipse IDE for Java EE Developers. Найбільш відомим web server є Apache Tomcat. Стверджується, що коректна робота вебзастосунку при розгортанні на Apache Tomcat забезпечує коректну роботу на усіх інших web

server або application server. Виходячи з цього рекомендується встановити зазначені програмні продукти.

Підготовка компонентів перед встановленням IntelliJ IDEA. Перед встановленням інтегрованого середовища розробки IntelliJ IDEA для розробки на Java на операційній системі macOS необхідно підготувати наступні компоненти описані нижче.

Homebrew – це менеджер пакетів для операційної системи macOS. Встановити Homebrew можна використовуючи наступної команди в терміналі:
`"/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`.

Крім того, до встановлення IDE та Tomcat слід встановити на комп'ютер необхідну версію JDK SE. Рекомендованою версією JDK SE на поточний момент є JDK 17. Всі більш пізніші версії JDK також забезпечують потрібний функціонал.

Java Development Kit (JDK) – це комплект розробника застосунків на мові Java. IntelliJ IDEA потребує встановлений JDK для компіляції та запуску Java програм. Тому рекомендується встановити JDK (від 17 версії і вище) від Oracle або OpenJDK. Перевірити чи встановлено JDK на системі можна виконавши команди в терміналі: `"java -version"`. Якщо JDK не встановлений, його можна завантажити з сайту Oracle за посиланням: <https://www.oracle.com/java/technologies/downloads/#java11> вибравши файл для відповідної операційної системи та архітектури процесора. Або встановити через Homebrew виконавши команди в терміналі: `"brew install openjdk"`.

Apache Maven – це засіб автоматизації роботи з програмними проектами. Maven використовується для керування та складання програм. Його можна встановити використовуючи команду Homebrew в терміналі `"brew install maven"` або завантажити з офіційного сайту Apache Maven <https://maven.apache.org/download.cgi> вибравши файл для відповідної операційної системи та архітектури процесора та притримуючись інструкції з встановлення: <https://maven.apache.org/install.html>.

Якщо описані вище компоненти вже встановлені на пристрої, то можна переходити до встановлення інтегрованого середовища розробки IntelliJ IDEA.

Встановлення IntelliJ IDEA. IntelliJ IDEA можна встановити виконавши кроки описані нижче.

Перейти на сторінку завантаження на офіційному сайті JetBrains <https://www.jetbrains.com/idea/download/> та вибрати файл для відповідної операційної системи.

Для виконання подальших завдань лабораторної роботи необхідно мати версію IntelliJ IDEA Ultimate – це платна версія з підтримкою різних фреймворків і інструментів для професійної розробки. Студенти та викладачі можуть отримати безкоштовну ліцензію на платну версію IntelliJ IDEA Ultimate перейшовши на сторінку <https://www.jetbrains.com/community/education/#students> та заповнивши форму за посиланням <https://www.jetbrains.com/shop/eform/students>.

Після завантаження інсталяційного файлу “*.dmg” для операційної системи macOS відкрити його.

Перетягнути значок IntelliJ IDEA до папки Applications.

Відкрити папку Applications та знайти IntelliJ IDEA.

Відкрити IntelliJ IDEA. Якщо програма відкривається вперше на після встановлення, macOS може попередити, що це програма, завантажена з інтернету. У вікні з попередженням натиснути “Open”.

Початкове налаштування IntelliJ IDEA можна виконати за кроками описаними нижче.

1 Після першого запуску програмою буде запропоновано імпортувати налаштування з попередньої версії або почати з нуля. Якщо дана програма раніше не використовувалась, то вибрати “Do not import settings”.

2 На екрані привітання можна вибрати тему оформлення (світлу чи темну).

3 Далі IntelliJ IDEA запропонує встановити додаткові плагіни або почати роботу без них (встановити плагіни можна і пізніше через налаштування).

4 Необхідно оновити або підтвердити наявність вже встановленого JDK у системі. Якщо JDK не встановлений, IntelliJ IDEA допоможе його встановити або виконайте інструкції описані у тексті роботи.

1.2 Встановлення вебсервера (web server) або сервера застосунків (application server) для розгортання вебзастосунків на Java

Рекомендовано використовувати Apache Tomcat через його легкість в опануванні, надійність, підтримці стандартів Java, а також його широке застосування в індустрії для розгортання вебзастосунків.

Apache Tomcat – це популярний веб-сервер і контейнер сервлетів, розроблений для запуску вебзастосунків на основі Java. Його основна мета – обробка динамічного контенту, зокрема виконання Java сервлетів та JavaServer Pages (JSP), що є важливою частиною Java веб-розробки.

Слід зазначити, що Apache Tomcat не установлюється без наявності JDK SE на комп'ютері.

Щоб встановити Apache Tomcat на macOS, можна скористатися інструкцією за посиланням <https://youtu.be/Gf7HP2SulqU?si=VK08WEivZSul2yTM> або виконати кроки описані нижче.

1 Перейти на офіційний сайт Apache Tomcat: <https://tomcat.apache.org/>.

2 У розділі “Download” вибрати останню стабільну версію Tomcat (на момент написання роботи це Tomcat 10.1.28). Знайти розділ “Binary Distributions” пункт “Core” та завантажити архів (файл “.tar.gz” для macOS). Приклад структури сторінки завантаження зображений на рис. 1 наведеному далі в тексті роботи.

3 Перейти у папку з завантаженим архівом скопіювати його та перемістити файл архіву до бажаної папки розташування. Рекомендованою папкою є “/Users/username”, де “username” – це ім'я користувача комп'ютера. Приклад розміщення файлу зображений на рис. 2 наведеному далі в тексті роботи.

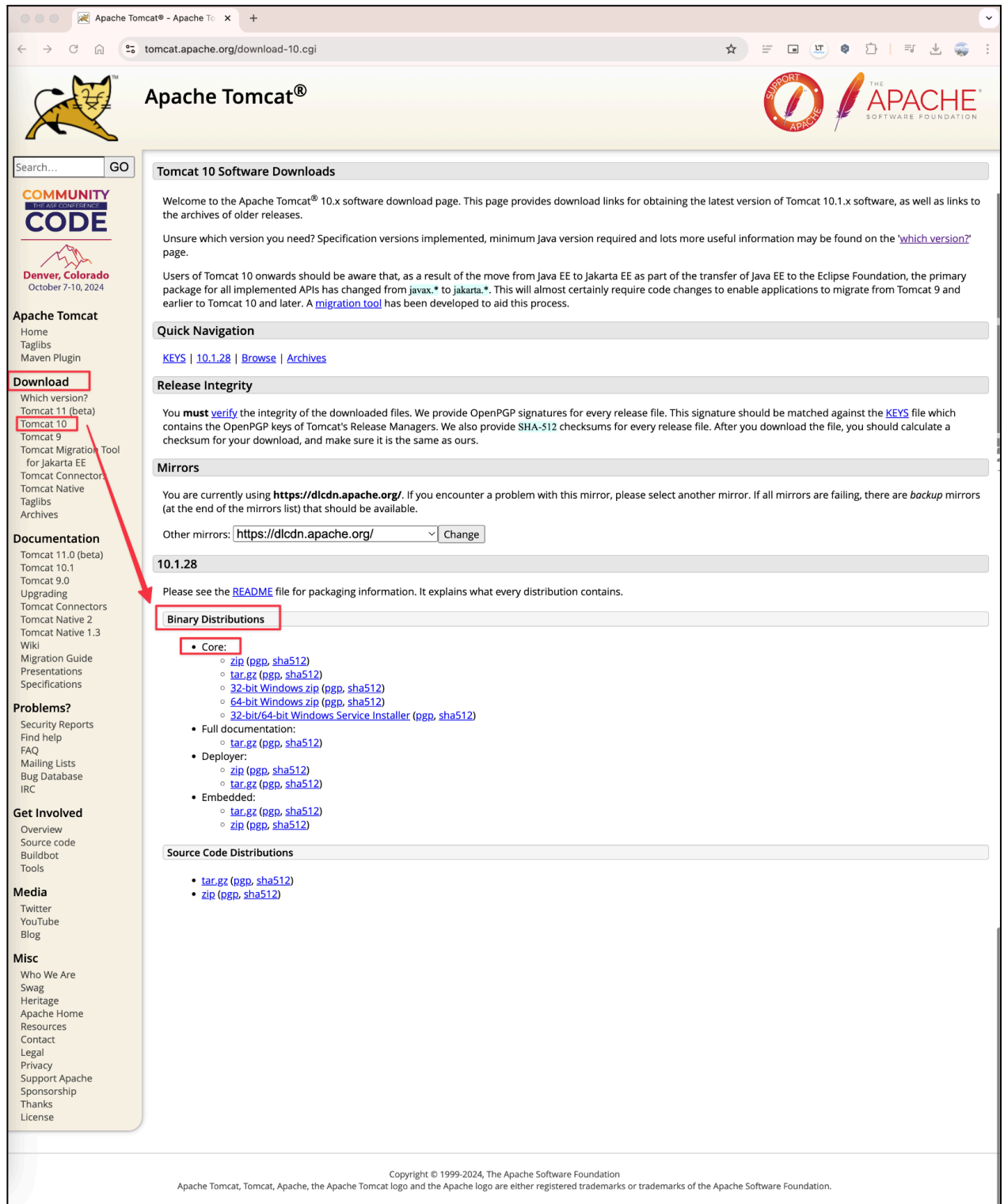


Рисунок 1 – Сторінка завантаження Apache Tomcat

4 Далі відкрити Термінал на комп'ютері і виконати команди для переходу до папки "bin" у папці з файлами Apache Tomcat та запуску серверу використовуючи файл `./startup.sh` – це файл команд мови сценаріїв, який містить комп'ютерну програму для запуску оболонкою Unix. Приклад

результату виконання команд зображень на рис. 3 наведеному далі в тексті роботи. Успішним результатом запуску сервера Apache Tomcat буде виведення повідомлення “Tomcat started.” у Терміналі.

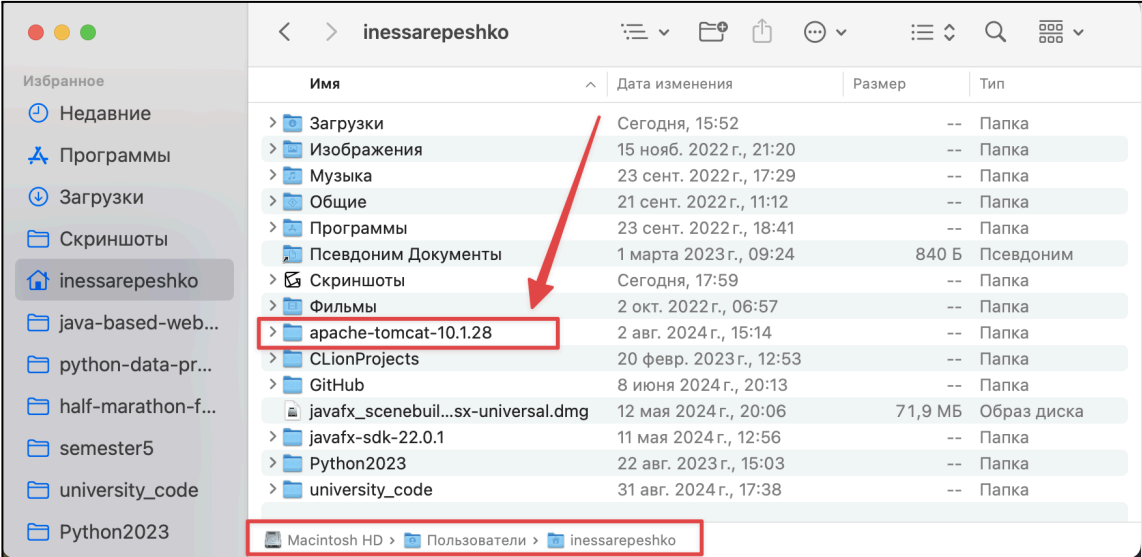


Рисунок 2 – Приклад розміщення файлу архіву Apache Tomcat

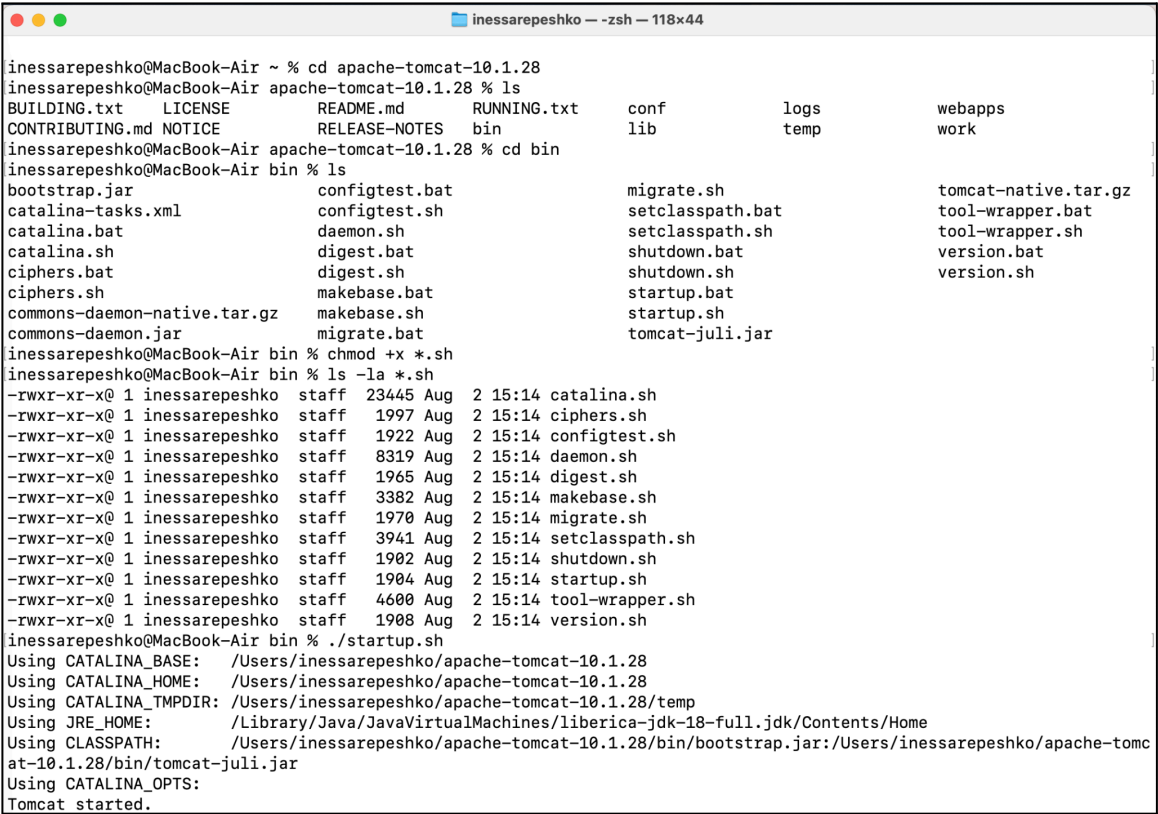


Рисунок 3 – Результат виконання команд для встановлення Apache Tomcat у Терміналі

5 Для перевірки успішного запуску сервера Apache Tomcat необхідно відкрити браузер і у адресному рядку ввести IP та порт на якому запущений сервер “localhost:8080” та натисніть “Enter”. Результатом має бути привітальна сторінка сервера Apache Tomcat приклад якої зображений на рис. 4.

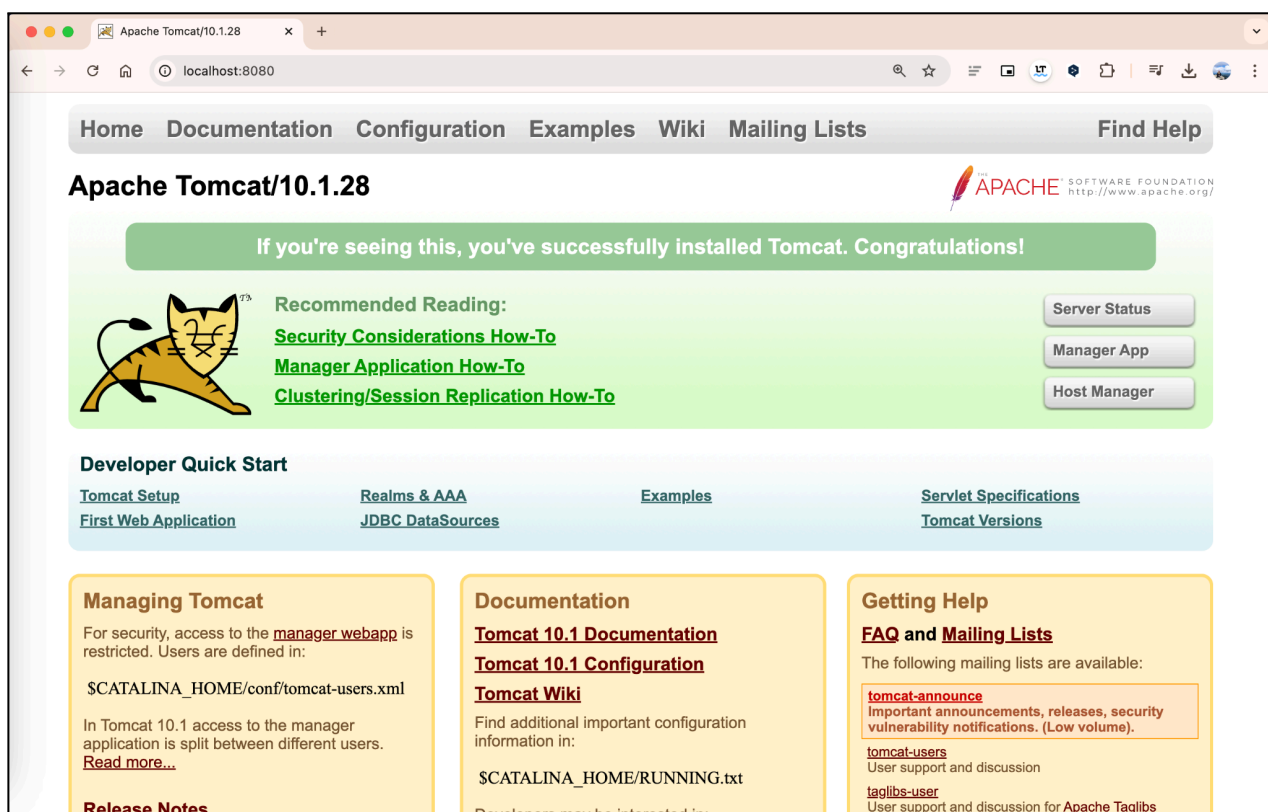


Рисунок 4 – Привітальна сторінка запущеного сервера Apache Tomcat за адресою “localhost:8080”

1.3 Створення та налаштування проєкту у встановленому інтегрованому середовищі розробки

Для подальшого створення вебзастосунку необхідно створити новий проєкт у встановленому інтегрованому середовищі розробки IntelliJ IDEA. Для цього потрібно запустити застосунок IntelliJ IDEA і натиснути “New Project” як зображено на рис. 5.

Далі необхідно встановити основні параметри проєкту відповідно до налаштувань відображених на рис. 6.

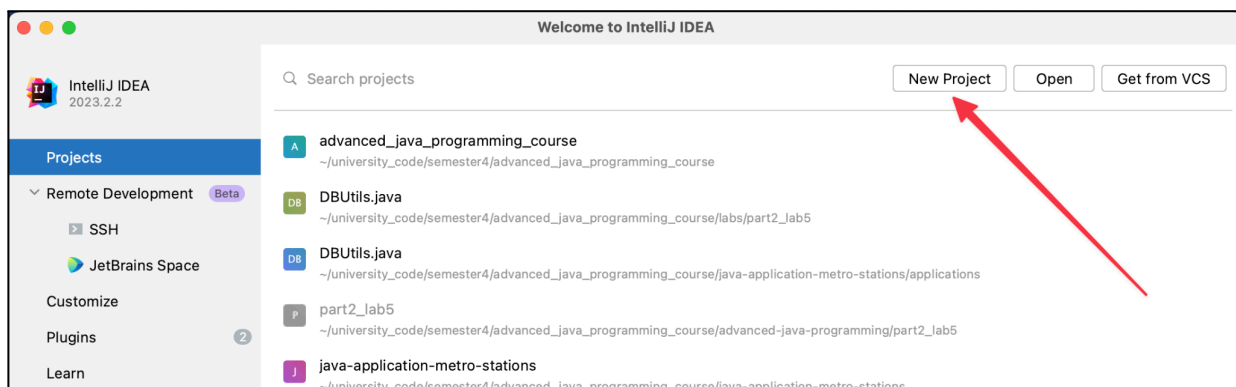


Рисунок 5 – Створення проєкту в IntelliJ IDEA

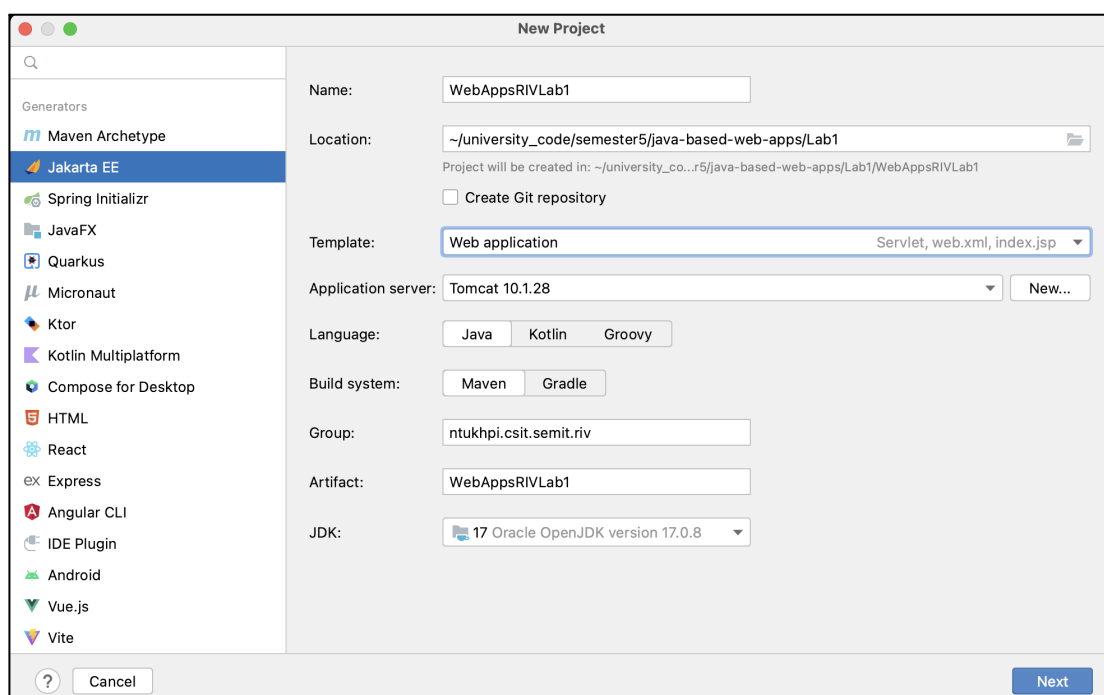


Рисунок 6 – Налаштування основних параметрів проєкту в IntelliJ IDEA

Також на сторінці налаштувань проєкту необхідно зазначити шлях до кореневого каталогу з встановленим сервером застосунку Apache Tomcat як зображено на рис. 7.

На сторінці налаштувань проєкту натиснути кнопку “Next”. В отриманому вікні слід встановити версію платформи та вказати залежності для проєкту відповідні тим, що зображені на рис. 8.

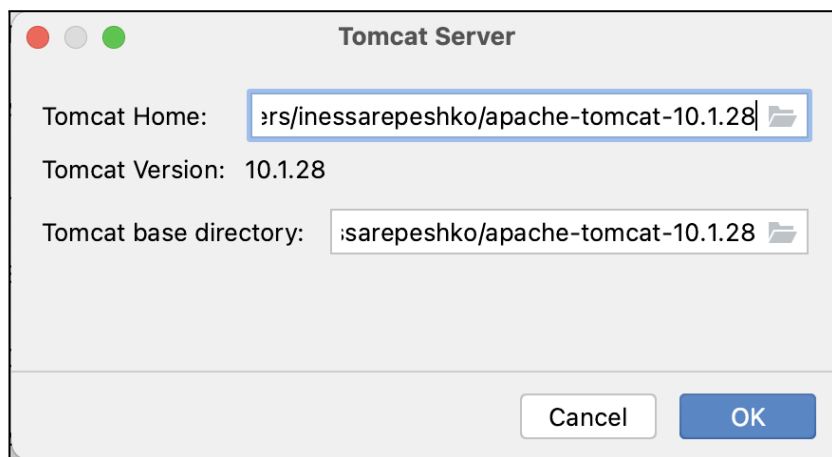


Рисунок 7 – Налаштування шляху до серверу застосунку Apache Tomcat

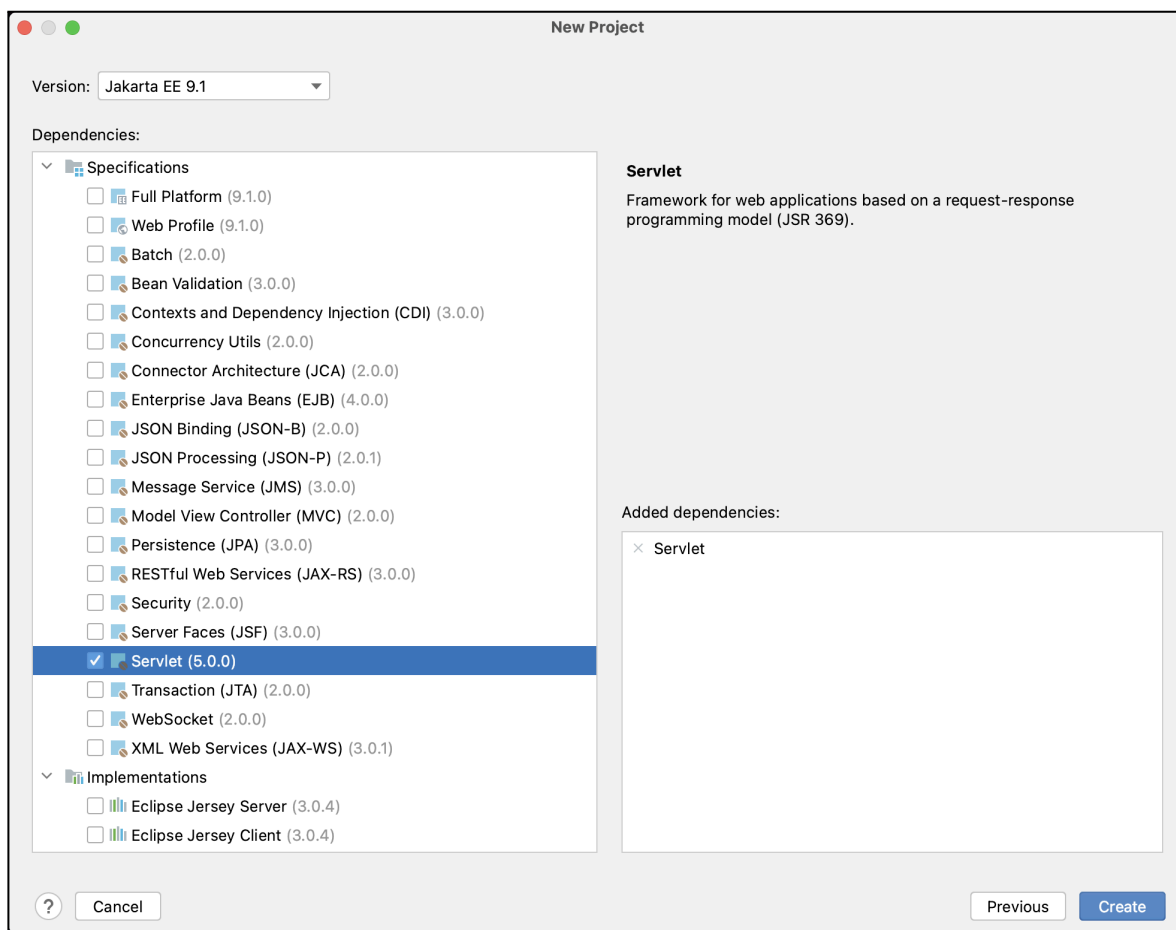


Рисунок 8 – Сторінка вибору платформи та залежностей проекту в IntelliJ IDEA

Після натискання кнопки “Create” створений проєкт буде відкритий у інтегрованому середовищі розробки IntelliJ IDEA і він матиме структуру відповідну до наведеної на рис. 9.

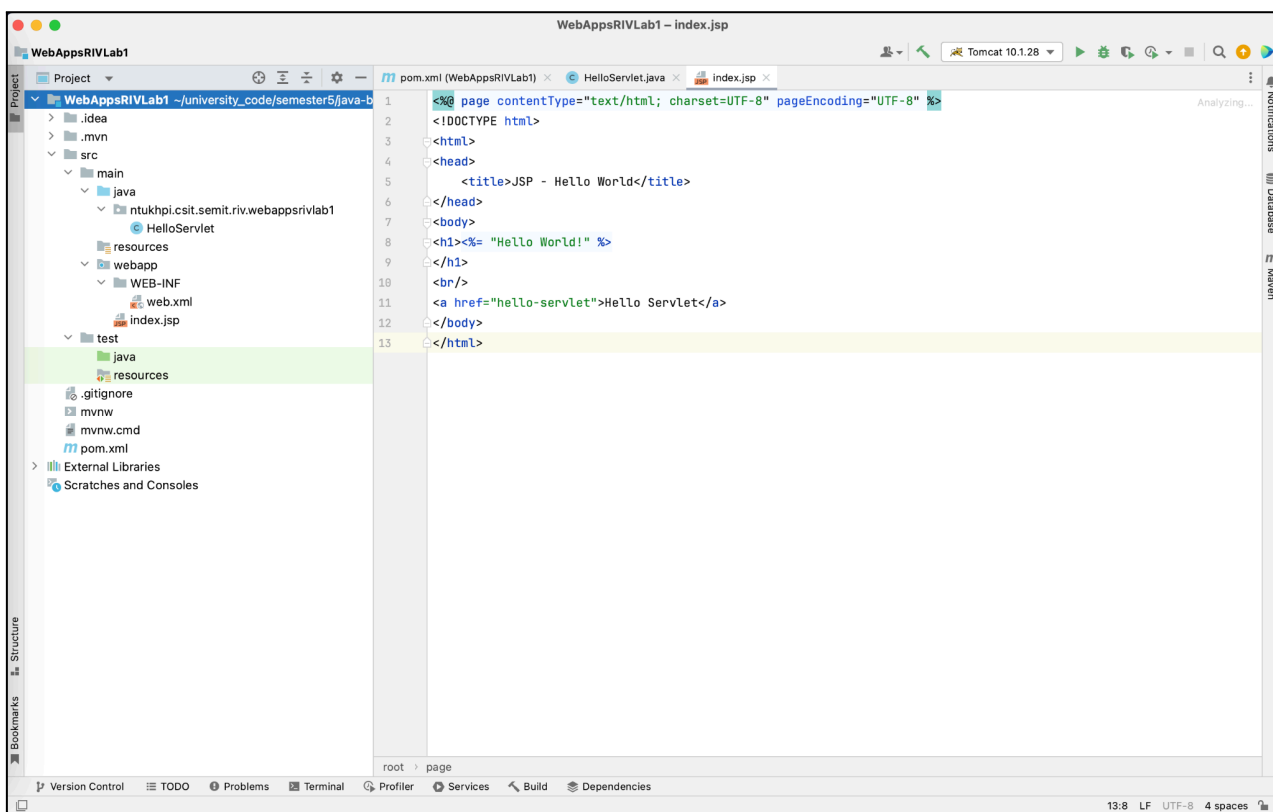


Рисунок 9 – Приклад структури створеного проєкту в IntelliJ IDEA

Представлені налаштування у тесті на рисунках 6-9 рекомендують параметри для роботи, але не виключають встановлення іншої комбінації. При цьому слід перевіряти сумісність компонентів, потрібних для коректної роботи проєкту. Наприклад, вебзастосунки, створені на основі патерну «Jakarta EE 9.1» не розгортаються на Tomcat 9.0.95.

1.4 Створення вебзастосунку у встановленому інтегрованому середовищі розробки та розгортання його на встановленому сервері

Створений вебзастосунок відображає на екрані наступні дані:

- текстовий напис “Hello, Web World!!!”;
- прізвище та ім’я студента;
- номер групи, в якій навчається студент;
- приналежність студента до департаменту;
- рік виконання роботи.

Результатом виконання даного завдання є реалізований проєкт “WebAppsRIVLab1_1” з використанням сервлетів та JSP для відображення описаних вище даних. Структура файлів проєкту відображена на рис. 10.

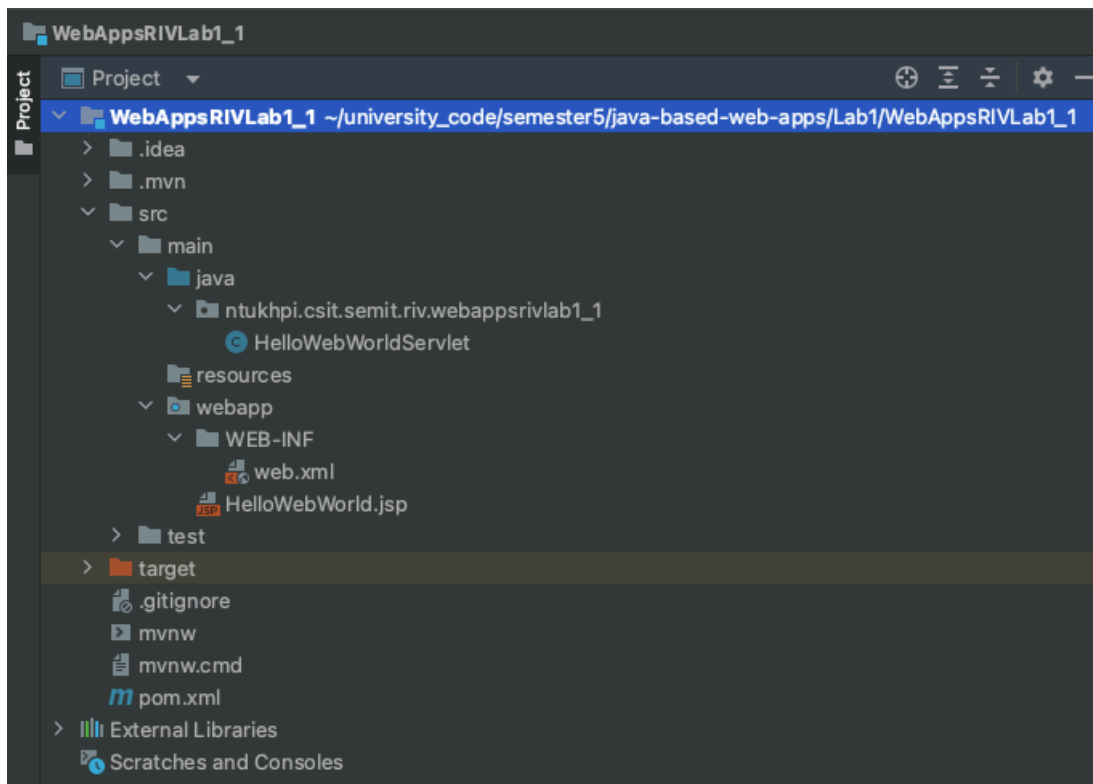


Рисунок 10 – Структура файлів проєкту “WebAppsRIVLab1_1”

Проект “WebAppsRIVLab1_1” складається з двох основних компонентів:

1 Сервлет HelloWebWorldServlet (представлений у Додатку А).

Сервлет HelloWebWorldServlet відповідає за обробку HTTP GET-запитів і відображення вмісту сторінки HelloWebWorld.jsp. Основні риси сервлету:

Маршрутизація: сервлет обробляє запити на два URL-адреси: “/hello-web-world” і кореневий шлях “/”. Це дозволяє отримати доступ до застосунку з різних шляхів.

Метод doGet(): обробляє GET-запити і включає вміст JSP-сторінки через RequestDispatcher. Таким чином, сервлет зберігає контроль над відповіддю і може продовжувати обробку після включення вмісту JSP.

Даний сервлет контролює логіку маршрутизації і відображення інформації через JSP.

2 JSP-сторінка `HelloWebWorld.jsp` (представлена у Додатку Б).

Дана JSP-сторінка, що відповідає за відображення даних про студента і загального повідомлення. Вона містить:

Текстовий напис “Hello, Web World!!!” у вигляді заголовку `<h1>`, стилізованого з використанням бібліотеки Bootstrap.

Блок з інформацією про студента, який відображає:

- прізвище та ім’я: “Repeshko Inessa”;
- номер групи: “CS-222a”;
- приналежність студента до департаменту: “SEMIT”;
- рік виконання роботи: “2024”.

Сторінка використовує Bootstrap для кращого форматування контенту та адаптивного дизайну. Контент поділений на секції з використанням CSS-класів Bootstrap, що робить його зручним для перегляду на різних пристроях.

На рис. 11 та 12 представлені результати роботи вебзастосунку реалізованого до завдання 1.4.

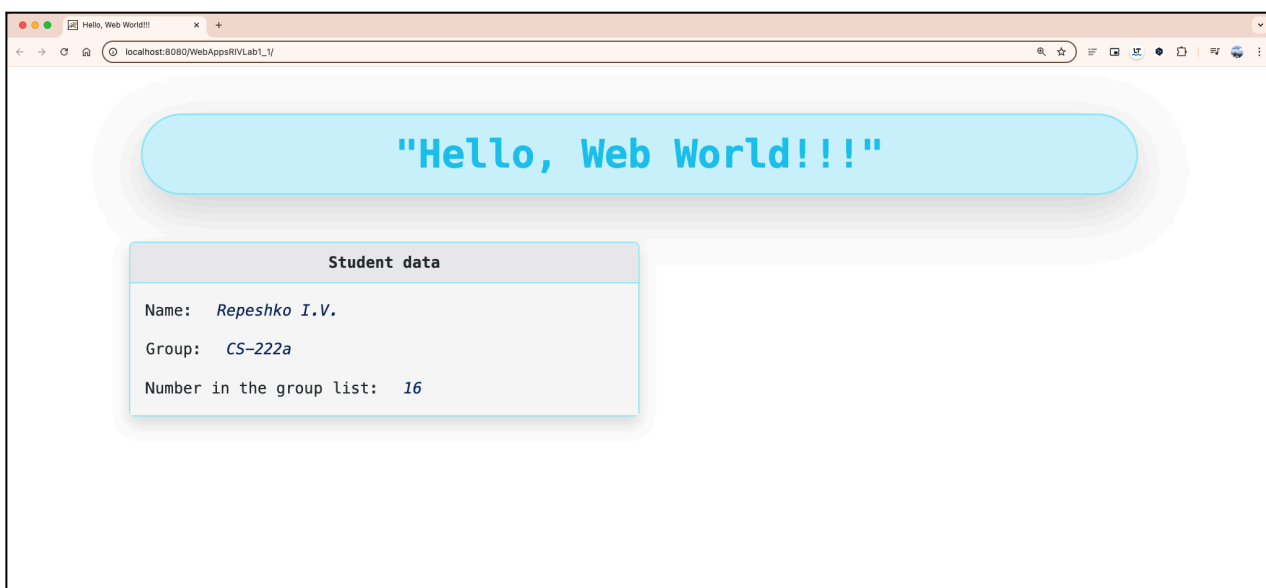


Рисунок 11 – Результат роботи вебзастосунку реалізованого до завдання 1.4 на кореневому шляху “/”

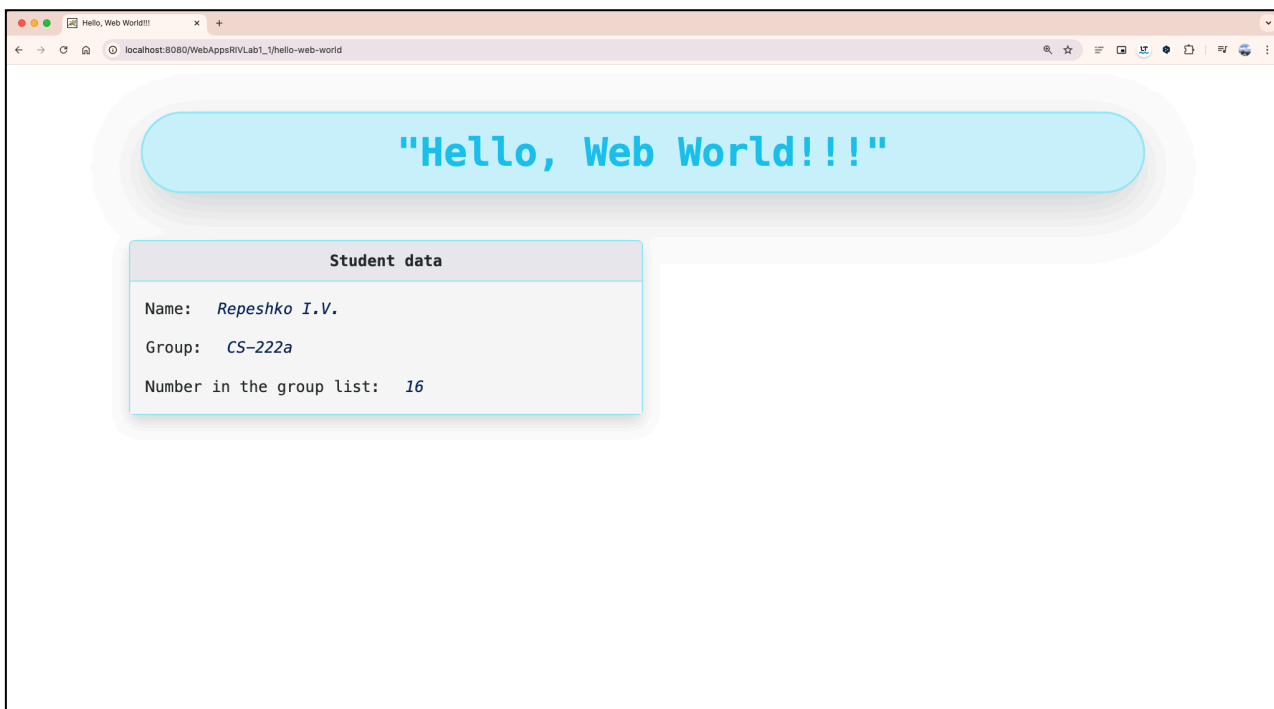


Рисунок 12 – Результат роботи вебзастосунку реалізованого до завдання 1.4 на шляху “/hello-web-world”

2 Розробка вебзастосунку, який дозволяє відобразити на веб-сторінці перелік співробітників деякої ІТ-компанії

Результатом виконання даного завдання є реалізований проєкт “WebAppsRIVLab1_2” на основі сервлетів Java, що відображає список співробітників у форматі HTML-таблиці. Структура файлів проєкту відображена на рис. 10.

Проєкт складається з наступних основних компонентів:

1 Клас Employee (представлений у Додатку В).

Клас Employee представляє сутність співробітника. Він містить наступні поля:

- id – ідентифікатор співробітника.
- name – ім'я співробітника.
- gender – стать співробітника (чоловік/жінка).
- birthday – дата народження співробітника у форматі LocalDate.
- salary – зарплата співробітника.

- `programLanguage` – основна мова програмування співробітника (тип `ProgramLanguages`).
- `importance` — рівень важливості співробітника (низький, середній, високий).

Клас також має методи для перевірки та парсингу введених даних (ім'я, стать, зарплата та інші), що дозволяє забезпечити коректність введених даних або використати значення за замовчуванням, якщо введення некоректне.

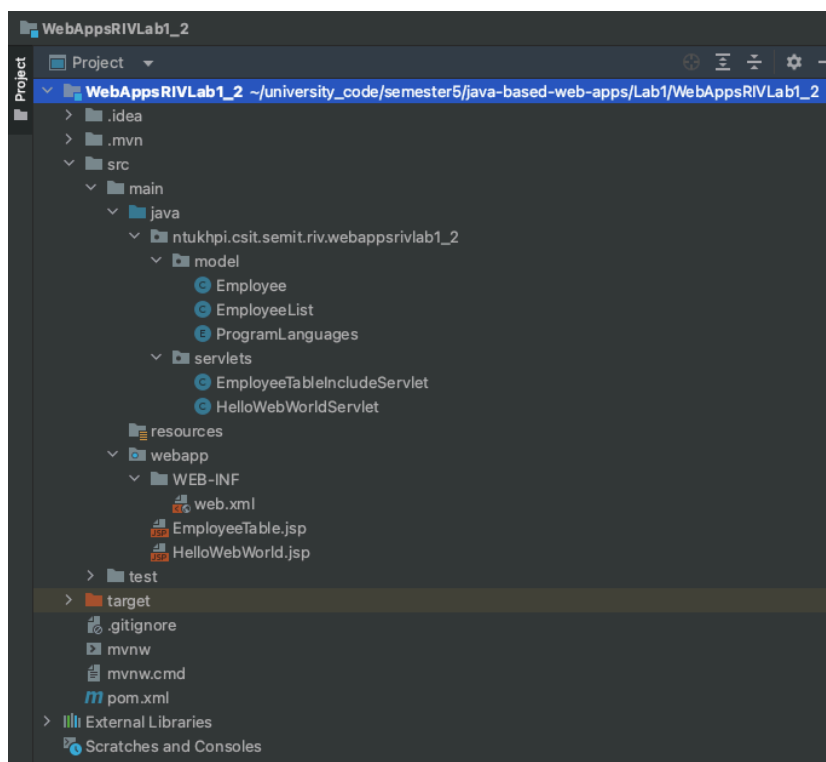


Рисунок 13 – Структура файлів проєкту “WebAppsRIVLab1_2” з реалізацією завдання 2

2 Клас `EmployeeList` (представлений у Додатку Г).

Даний клас слугує контейнером для об'єктів `Employee` і керує списком співробітників. Основні методи класу дозволяють додавати нових співробітників до списку та отримувати доступ до списку для відображення його на JSP-сторінках.

3 Перелік `ProgramLanguages` (представлений у Додатку Д).

Перелік ProgramLanguages містить список доступних мов програмування, які може знати співробітник. Серед можливих значень: Java, Python, C++, JavaScript та інші.

4 Сервлет EmployeeTableIncludeServlet (представлений у Додатку Е).

Сервлет EmployeeTableIncludeServlet відповідає за обробку HTTP-запитів, формування списку співробітників і передавання цього списку на JSP-сторінку. Він отримує список об'єктів Employee з класу EmployeeList, після чого передає його на JSP-сторінку для відображення у вигляді таблиці.

5 Сервлет HelloWebWorldServlet (представлений у Додатку А).

Сервлет HelloWebWorldServlet відповідає за обробку HTTP GET-запитів і відображення вмісту сторінки HelloWebWorld.jsp. Даний сервлет контролює логіку маршрутизації і відображення інформації через JSP.

6 Сторінка HelloWebWorld.jsp (представлена у Додатку Ж).

Дана JSP-сторінка служить точкою входу для користувача та відображає текстовий заголовок “Hello, Web World!!!” і картку з інформацією про студента яка містить дані: прізвище та ім'я, номер групи, і порядковий номер у списку. Крім цього, на сторінці відображається картка з інформацією про таблицю співробітників та метод include(). Кнопка “Show” дозволяє перейти до сторінки з таблицею співробітників, що включена з використанням методу include() класу RequestDispatcher.

7 Сторінка EmployeeTable.jsp (представлена у Додатку И).

JSP-сторінка EmployeeTable.jsp відображає список співробітників у вигляді HTML-таблиці. В таблиці виводяться такі поля, як Id, ім'я, стать, дата народження, зарплата, мова програмування та рівень важливості для компанії. Використовується динамічна генерація таблиці з використанням циклу, що проходить через кожен елемент списку співробітників.

Всі сторінки вебзастосунку використовують Bootstrap для кращого форматування контенту та адаптивного дизайну. Контент поділений на секції з використанням CSS-класів Bootstrap, що робить його зручним для перегляду на різних пристроях.

Далі в тексті роботи на рис. 14, 15 та 16 представлені результати роботи вебзастосунку реалізованого до завдання 2.

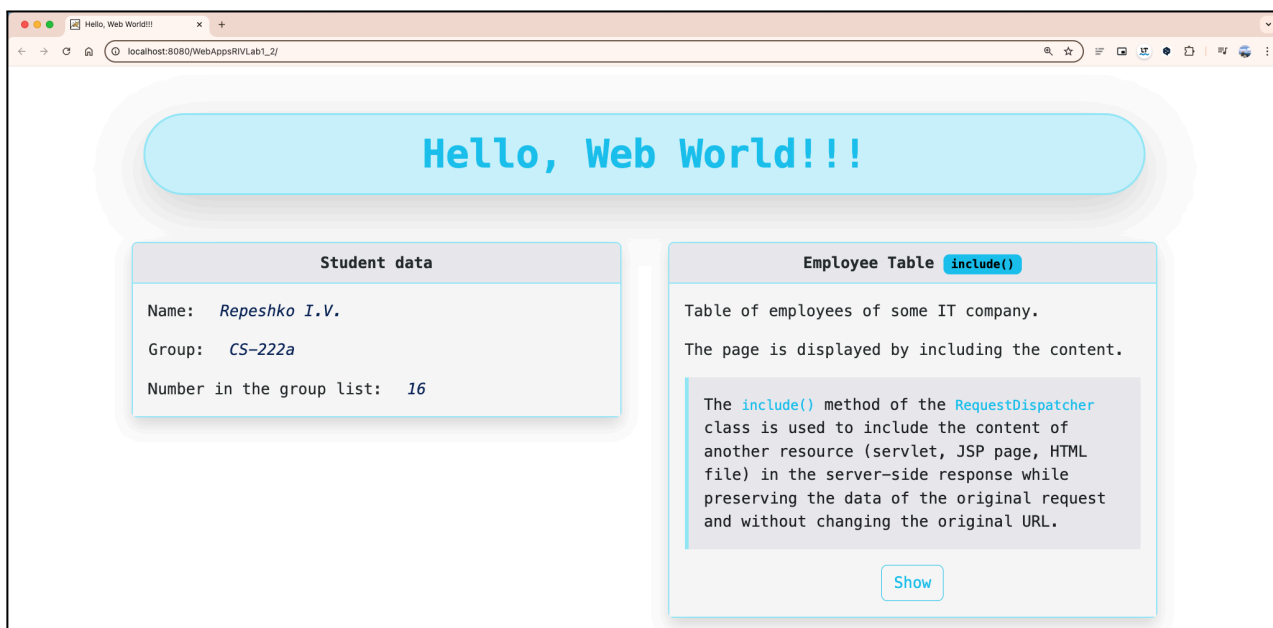


Рисунок 14 – Результат роботи вебзастосунку реалізованого до завдання 2 на кореневому шляху “/”

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/WebAppsRIVLab1_2/employee-table-by-include-method'. The main content area features a large light blue rounded rectangle with the text 'Employee Table'. Below this, there is a table with 13 rows of employee data. The table has the following columns: Id, Name, Gender, Birthday, Salary, UAH, Program Language, and Importance. The data is as follows:

Id	Name	Gender	Birthday	Salary, UAH	Program Language	Importance
1	Alice Johnson	female	12.03.1985	120000,00	JavaScript	high
2	Bob Smith	male	18.06.1990	95000,00	TypeScript	medium
3	Catherine Brown	female	25.11.1978	145000,00	Python	medium
4	David Wilson	male	04.02.1995	85000,00	Java	low
5	Emma Davis	female	30.07.1983	100000,00	C#	medium
6	Frank Miller	male	20.01.2000	70000,00	PHP	low
7	Grace Lee	female	15.05.1988	110000,00	JavaScript	high
8	Henry White	male	22.09.1993	90000,00	TypeScript	low
9	Isabella Green	female	07.08.1997	80000,00	Python	low
10	James Black	male	13.12.1981	115000,00	Java	high
11	Olivia Thompson	female	03.04.1999	95000,00	C#	medium
12	William Garcia	male	29.10.1975	130000,00	PHP	high
13	Name Surname	male	01.01.2006	8000,00	JavaScript	low

At the bottom of the page is a 'Go Back' button.

Рисунок 15 – Результат роботи вебзастосунку реалізованого до завдання 2 після натискання кнопки “Show” у картці “Employee Table include()” (на шляху “employee-table-by-include-method”)

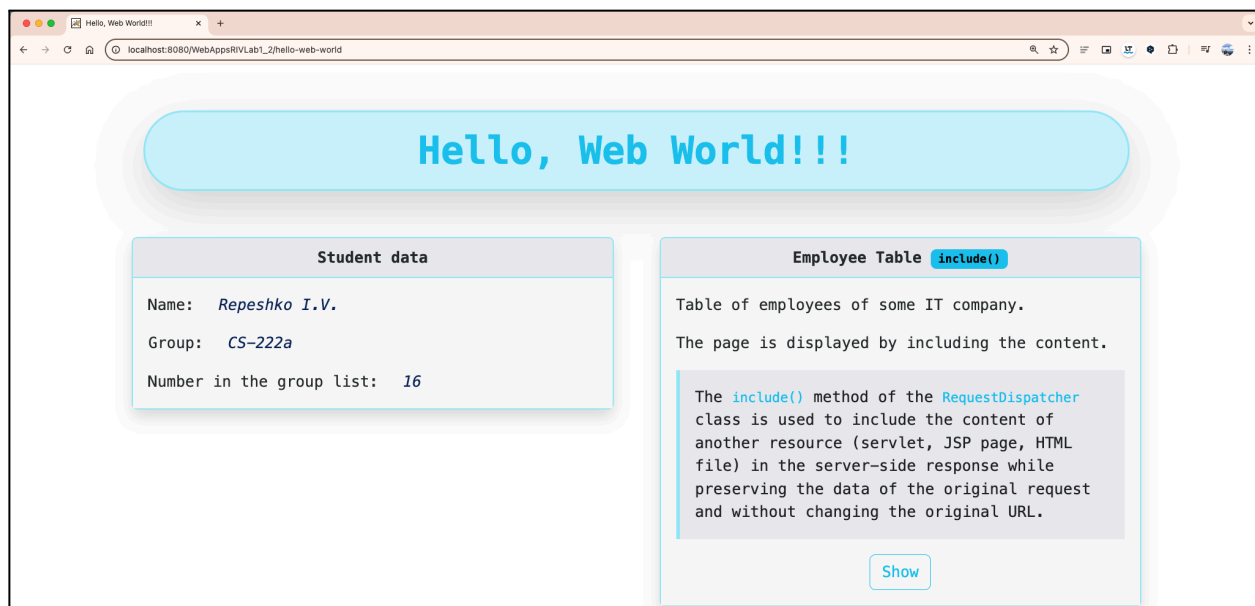


Рисунок 16 – Результат роботи вебзастосунку реалізованого до завдання 2 після натискання кнопки “Go Back” та сторінці “Employee Table” (на шляху “hello-web-world”)

3 Модифікація проєкту, створеного після виконання завдання 2, шляхом додавання функціоналу переходу на готову сторінку із сервлету, використовуючи команди `forward()` та `sendRedirect()`

Результатом виконання даного завдання є модифікований проєкт “WebAppsRIVLab1_2” реалізований у попередньому завданні, шляхом створення сервлетів Java з використанням методів `forward()` та `sendRedirect()`, що відображають список співробітників у форматі HTML-таблиці. Структура файлів модифікованого проєкту відображена на рис. 17.

В результаті виконання поточного задання у проєкт “WebAppsRIVLab1_2” було додано або змінено наступні компоненти:

1 Сервлет `EmployeeTableForwardServlet` (представлений у Додатку К).

Даний сервлет відповідає за пересилання запиту клієнта на сторінку `EmployeeTable.jsp` на стороні сервера застосовуючи метод `forward()` класу `RequestDispatcher` зі збереженням оригінальних даних запиту та без зміни URL-адреси на стороні користувача.

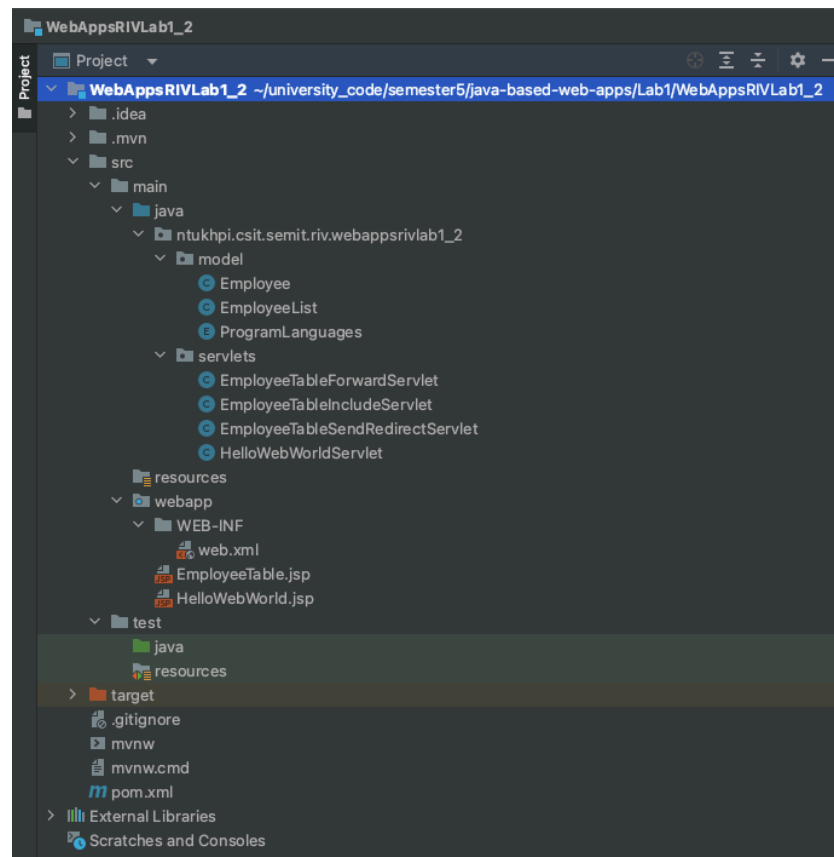


Рисунок 17 – Структура файлів модифікованого проєкту “WebAppsRIVLab1_2” з реалізацією завдання 3

2 Сервлет EmployeeTableSendRedirectServlet (представлений у Додатку Л).

Даний сервлет відповідає за перенаправлення запиту клієнта на сторінку EmployeeTable.jsp (внутрішній ресурс) на стороні клієнта застосовуючи метод `sendRequest()` класу `HttpServletResponse` з передачею даних з використанням GET-параметрів в URL-адресі та зі зміною URL-адреси на стороні користувача.

3 Сторінка HelloWebWorld.jsp (представлена у Додатку М).

Дана JSP-сторінка служить точкою входу для користувача та відображає текстовий заголовок “Hello, Web World!!!” і картку з інформацією про студента яка містить дані: прізвище та ім'я, номер групи, і порядковий номер у списку. Крім цього, на сторінці відображаються ще три картки:

– з інформацією про таблицю співробітників та метод `include()`, а кнопка “Show” у ній дозволяє перейти до сторінки з таблицею співробітників, що включена із застосуванням методу `include()` класу `RequestDispatcher`;

– з інформацією про таблицю співробітників та метод `forward()`, а кнопка “Show” у ній дозволяє перейти до сторінки з таблицею співробітників, застосовуючи пересилання запиту методом `forward()` класу `RequestDispatcher`;

– з інформацією про таблицю співробітників та метод `sendRequest()`, а кнопка “Show” у ній дозволяє перейти до сторінки з таблицею співробітників, застосовуючи перенаправлення запиту методом `sendRequest()` класу `HttpServletResponse`.

4 Сторінка `EmployeeTable.jsp` (представлена у Додатку Н).

JSP-сторінка `EmployeeTable.jsp` відображає список співробітників у вигляді HTML-таблиці. В таблиці виводяться такі поля, як `Id`, ім'я, стать, дата народження, зарплата, мова програмування та рівень важливості. Використовується динамічна генерація таблиці з використанням циклу, що проходить через кожен елемент списку співробітників. Дані про список співробітників отримуються з GET-параметрів або атрибутів запиту.

Всі сторінки вебзастосунку використовують фреймворк `Bootstrap` для кращого форматування контенту та адаптивного дизайну. Використання `Bootstrap` у цьому проєкті надало потужні можливості для швидкої та ефективної розробки інтерфейсів з використанням готових компонентів і стилів. Контент сторінок поділений на секції з використанням CSS-класів `Bootstrap`, що робить його зручним для перегляду на різних пристроях. Використання готових елементів, таких як кнопки, форми, навігаційні панелі, таблиці, модальні вікна та інші інтерфейсні компоненти значно скоротило час розробки. Інтеграція цих компонентів дозволило створити сучасні, інтерактивні та естетично привабливі веб-сторінки, зберігаючи консистенцію стилів у всьому проєкті.

На рис. 18-22 представлені результати роботи вебзастосунку реалізованого до завдання 3.



Рисунок 18 – Результат роботи вебзастосунку реалізованого до завдання 3 на кореневому шляху “/”

На домашній сторінці реалізованого вебзастосунку (рис. 18) є можливість переходу до сторінки з таблицею працівників деякої ІТ компанії по кнопці “Show” з використанням різних методів передачі запитів:

1 Метод `include()` класу `RequestDispatcher`.

Даний метод використовується для передачі запиту від одного сервлета або JSP до іншого ресурсу (сервлета, JSP або HTML-сторінки) на сервері безпосередньо. Запит обробляється повністю на стороні сервера, і клієнт не знає про цю передачу, оскільки URL у браузері залишається незмінним.

Employee Table

Id	Name	Gender	Birthday	Salary, UAH	Program Language	Importance
1	Alice Johnson	female	12.03.1985	120000,00	JavaScript	high
2	Bob Smith	male	18.06.1990	95000,00	TypeScript	medium
3	Catherine Brown	female	25.11.1978	145000,00	Python	medium
4	David Wilson	male	04.02.1995	85000,00	Java	low
5	Emma Davis	female	30.07.1983	100000,00	C#	medium
6	Frank Miller	male	20.01.2000	70000,00	PHP	low
7	Grace Lee	female	15.05.1988	110000,00	JavaScript	high
8	Henry White	male	22.09.1993	90000,00	TypeScript	low
9	Isabella Green	female	07.08.1997	80000,00	Python	low
10	James Black	male	13.12.1981	115000,00	Java	high
11	Olivia Thompson	female	03.04.1999	95000,00	C#	medium
12	William Garcia	male	29.10.1975	130000,00	PHP	high
13	Name Surname	male	01.01.2006	8000,00	JavaScript	low

[Go Back](#)

Рисунок 19 – Результат роботи вебзастосунку реалізованого до завдання 3 після натискання кнопки “Show” у картці “Employee Table include()” (на шляху “/employee-table-by-include-method”)

Employee Table

Id	Name	Gender	Birthday	Salary, UAH	Program Language	Importance
1	Alice Johnson	female	12.03.1985	120000,00	JavaScript	high
2	Bob Smith	male	18.06.1990	95000,00	TypeScript	medium
3	Catherine Brown	female	25.11.1978	145000,00	Python	medium
4	David Wilson	male	04.02.1995	85000,00	Java	low
5	Emma Davis	female	30.07.1983	100000,00	C#	medium
6	Frank Miller	male	20.01.2000	70000,00	PHP	low
7	Grace Lee	female	15.05.1988	110000,00	JavaScript	high
8	Henry White	male	22.09.1993	90000,00	TypeScript	low
9	Isabella Green	female	07.08.1997	80000,00	Python	low
10	James Black	male	13.12.1981	115000,00	Java	high
11	Olivia Thompson	female	03.04.1999	95000,00	C#	medium
12	William Garcia	male	29.10.1975	130000,00	PHP	high
13	Name Surname	male	01.01.2006	8000,00	JavaScript	low

[Go Back](#)

Рисунок 20 – Результат роботи вебзастосунку реалізованого до завдання 3 після натискання кнопки “Show” у картці “Employee Table forward()” (на шляху “/employee-table-by-forward-method”)

Id	Name	Gender	Birthday	Salary, UAH	Program Language	Importance
1	Alice Johnson	female	12.03.1985	120000,00	JavaScript	high
2	Bob Smith	male	18.06.1990	95000,00	TypeScript	medium
3	Catherine Brown	female	25.11.1978	145000,00	Python	medium
4	David Wilson	male	04.02.1995	85000,00	Java	low
5	Emma Davis	female	30.07.1983	100000,00	C#	medium
6	Frank Miller	male	20.01.2000	70000,00	PHP	low
7	Grace Lee	female	15.05.1988	110000,00	JavaScript	high
8	Henry White	male	22.09.1993	90000,00	TypeScript	low
9	Isabella Green	female	07.08.1997	80000,00	Python	low
10	James Black	male	13.12.1981	115000,00	Java	high
11	Olivia Thompson	female	03.04.1999	95000,00	C#	medium
12	William Garcia	male	29.10.1975	130000,00	PHP	high
13	Name Surname	male	01.01.2006	8000,00	JavaScript	low

[Go Back](#)

Рисунок 21 – Результат роботи вебзастосунку реалізованого до завдання 3 після натискання кнопки “Show” у картці “Employee Table sendRequest()” (на шляху “/EmployeeTable.jsp?employeeIds=1,2,3,4,5,6,7,8,9,10,11,12,13”)

У даній реалізації він використовується для динамічного включення JSP сторінки EmployeeTable.jsp.

2 Метод forward() класу RequestDispatcher.

Метод forward() класу RequestDispatcher в сервлетах використовується для передачі керування від одного сервлета або JSP-сторінки до іншого ресурсу на сервері (іншого сервлета, JSP чи статичного файлу). Цей метод виконує внутрішню переадресацію, не змінюючи URL-адресу в адресному рядку браузера.

3 Метод sendRedirect() класу HttpServletResponse.

Метод sendRedirect() виконує перенаправлення на інший URL-адресу. На відміну від forward(), це перенаправлення виконується на стороні клієнта. Браузер отримує статус 302 (Found) від сервера разом з новим URL, і потім робить новий запит на цю адресу. Через це клієнт бачить зміну в URL.

У даній реалізації цей метод використовується для переспрямування користувача на JSP сторінку EmployeeTable.jsp у рамках цієї ж програми з повідомленням клієнта про перенаправлення.

Результати переходів на відповідні сторінки після натискання кнопок “Show” на домашній сторінці відображені на рис. 19-21 вище в тексті роботи.

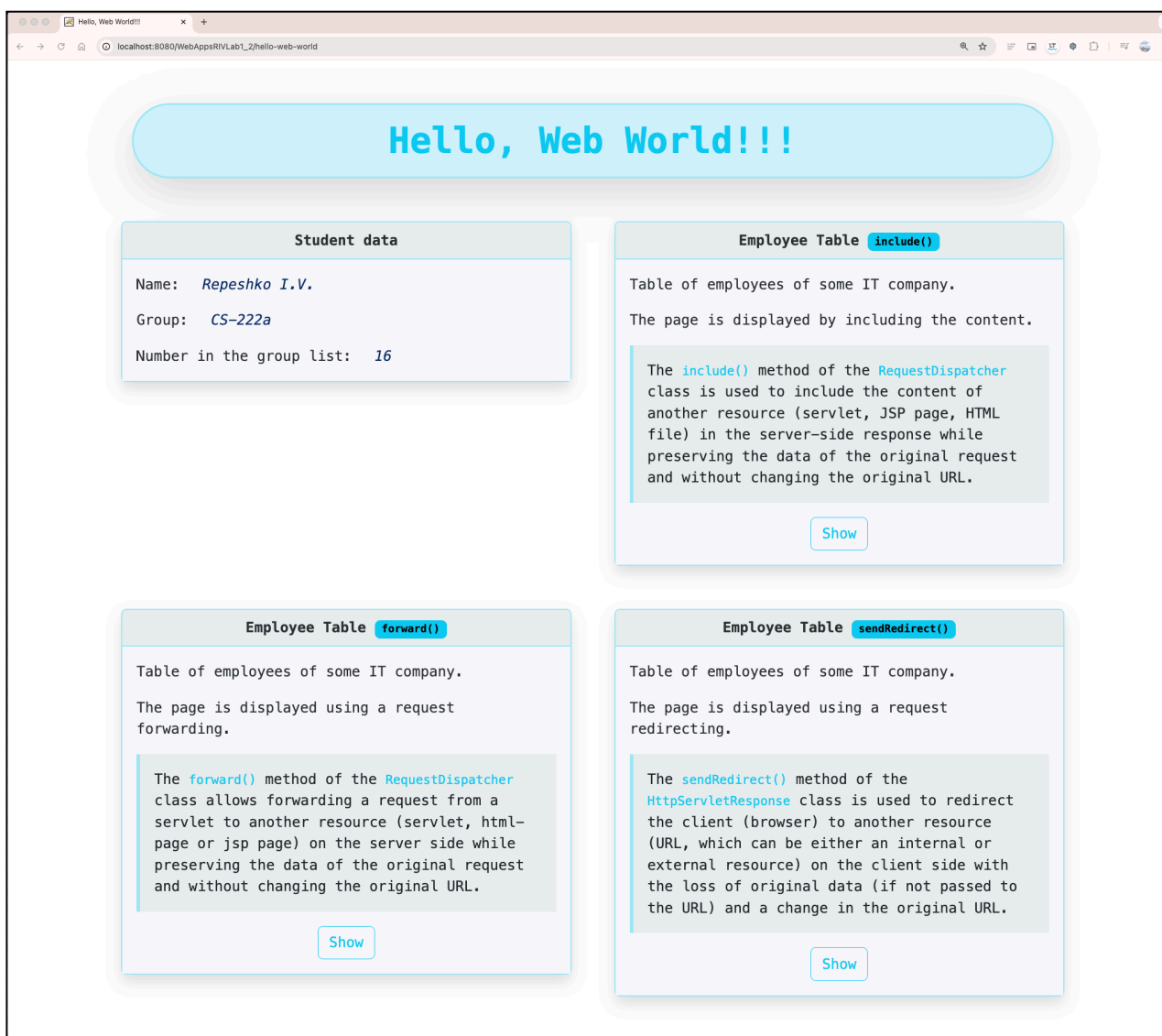


Рисунок 22 – Результат роботи вебзастосунку реалізованого до завдання 3 після натискання кнопки “Go Back” та сторінці “Employee Table” (з будь-яким методом відображення таблиці з трьох реалізованих) (на шляху “hello-web-world”)

4 Розробка вебзастосунку, який містить сторінку автентифікації, головну сторінку та сторінку з таблицею зі списком співробітників

Розроблений вебзастосунок, містить наступний функціонал:

- відображення стартової сторінки із полями для вводу логіну та паролю, після вводу правильної пари відкриває головну сторінку застосунку;
- відображення на головній сторінці меню, що дозволяє перейти до сторінки JSP із списком співробітників у вигляді таблиці без застосування можливостей Expression Language (EL) та Java Standard Tag Library (JSTL).

Результатом виконання даного завдання є реалізований проєкт “WebAppsRIVLab1_4”, що містить наступні сторінки:

1) сторінка автентифікації користувача: отримання логіну і паролю від користувача через форму та відправка запиту методом POST;

2) сторінка помилки авторизації: інформування користувача про помилку авторизації через невалідні облікові дані з кнопкою для повернення на сторінку автентифікації;

3) домашня сторінка містить наступні елементи:

- загальна навігаційна панель сайту у верхньому колонтитулі з клікабельними посиланнями на різні сторінки сайту;
- привітальний заголовок сторінки;
- контент сторінки з даними про студента та різними методами переходу до сторінки з таблицею з даними працівників деякої ІТ компанії у вигляді карток з кнопкою для переходу до таблиці;
- загальний футер з інформацією про розробника, організацію та її контакти;

4) сторінка з таблицею з даними працівників деякої ІТ компанії містить наступні елементи:

- навігаційна панель сайту у верхньому колонтитулі з клікабельними посиланнями на різні сторінки сайту;
- заголовок назви таблиці;
- таблиця з даними працівників;

- кнопка для повернення на домашню сторінку;
- інформація про розробника, організацію та її контакти у нижньому колонтитулі.

Структура файлів реалізованого проєкту відображена на рис. 23.

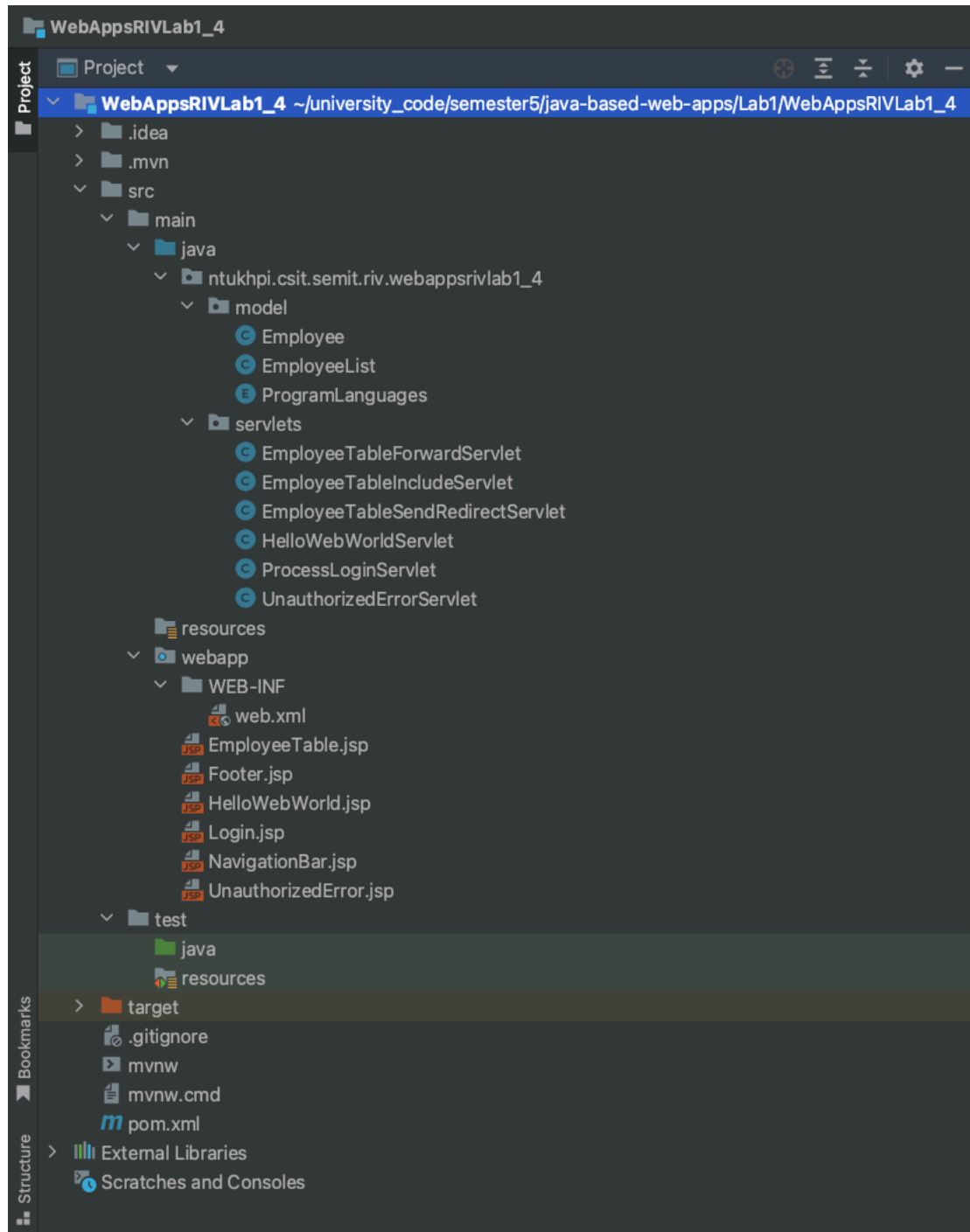


Рисунок 23 – Структура файлів проєкту “WebAppsRIVLab1_4” з реалізацією завдання 4

В результаті виконання поточного задання у проєкт “WebAppsRIVLab1_4” було додано або змінено наступні компоненти з проєкту реалізованого до попередніх завдань:

1 Сервлет `ProcessLoginServlet.java` (представлений у Додатку П).

Сервлет відповідає за авторизацію користувачів. Він отримує інформацію, введену користувачем під час входу (логін користувача та пароль), перевіряє її і виконує відповідну дію на основі результату (успішний вхід чи помилка). У разі успішної авторизації користувач перенаправляється на домашню сторінку, інакше — на сторінку з повідомленням про помилку авторизації.

2 Сервлет `UnauthorizedErrorServlet.java` (представлений у Додатку Р).

Даний сервлет керує відображенням сторінки помилки у випадку, коли користувач намагається отримати доступ до ресурсу з невалідними автентифікаційними даними і не проходить авторизацію. Він перенаправляє користувача на відповідну JSP сторінку з повідомленням про помилку та пропонує повернутися на головну сторінку для повторного входу.

3 Сторінка `Login.jsp` (представлена у Додатку С).

Сторінка `Login.jsp` містить форму для автентифікації (введення логіна і пароля), що забезпечує користувачам можливість автентифікуватися в системі. Форма надсилає введені автентифікаційні дані на сервлет для авторизації.

4 Сторінка `UnauthorizedError.jsp` (представлена у Додатку Т).

Дана сторінка відображає повідомлення про помилку авторизації. Якщо користувач не вказав валідні дані для автентифікації, ця сторінка надає йому повідомлення про те, що доступ заборонено (помилка 401). Сторінка також містить посилання для повернення на головну сторінку для повторної автентифікації.

5 Сторінка `HelloWebWorld.jsp` (представлена у Додатку У).

Сторінка `HelloWebWorld.jsp` є домашньою сторінкою вебзастосунку та виводить привітальне повідомлення “Hello Web World!” з логіном користувача з можливістю подальшого переходу по навігаційних посиланнях. Вона містить картку інформації про студента та час отримання доступу до домашньої

сторінки. А також на ній присутні картки з трьома методами переходу до сторінки `EmployeeTable.jsp` з таблицею з даними робітників деякої ІТ компанії. На початку сторінки включається загальна панель навігації з файлу `NavigationBar.jsp`, а наприкінці сторінки включається загальний футер з файлу `Footer.jsp`.

6 Сторінка `NavigationBar.jsp` (представлена у Додатку Ф).

Цей файл формує інтерфейс меню, який користувач може використовувати для навігації по вебзастосунку, що містить посилання на інші сторінки вебзастосунку, доступні користувачу після успішної авторизації. Цей компонент повторно використовується в різних частинах вебзастосунку.

7 Сторінка `Footer.jsp` (представлена у Додатку Х).

`Footer.jsp` відповідає за виведення футеру сторінки, де міститься інформація про розробника та його приналежність до компанії та контакти для зв'язку. Цей компонент повторно використовується в різних частинах вебзастосунку.

8 Сторінка `EmployeeTable.jsp` (представлена у Додатку Ц).

У цьому файлі генерується HTML-таблиця, яка містить інформацію про співробітників деякої ІТ компанії (ім'я, стать, дата народження, зарплата, мова програмування, важливість для компанії). Таблиця будується на основі даних, отриманих із сервлета або іншого джерела. На початку сторінки включається загальна панель навігації з файлу `NavigationBar.jsp`, а наприкінці сторінки включається загальний футер з файлу `Footer.jsp`.

9 Конфігураційний файл `web.xml` (представлений у додатку ІІІ).

Файл `web.xml` є конфігураційним файлом вебзастосунку, який визначає параметри ініціалізації та поведінку сервлетів і JSP-сторінок. У цьому файлі налаштовано список файлів-привітань (`welcome-file-list`), які автоматично відкриваються під час доступу до кореневої директорії застосунку — це `index.jsp`, `index.html` та `Login.jsp`. Також у файлі вказано конфігурацію для JSP-сторінок (`jsp-config`), де відзначено, що вирази мови EL ігноруються для сторінок, що відповідають шаблону `/webapp/*.jsp` для забезпечення вимоги

завдання про неможливість використання EL та JSTL. Крім того, файл визначає сторінку обробки помилок для коду помилки 401 (UnauthorizedError.jsp), яка буде відображатися у випадку неавторизованого доступу.

Всі сторінки вебзастосунку використовують Bootstrap для кращого форматування контенту та адаптивного дизайну. Контент домашньої сторінки вебзастосунку поділений на секції з використанням CSS-класів Bootstrap, що робить його зручним для перегляду на різних пристроях.

На рис. 24-30 представлені результати роботи вебзастосунку реалізованого відповідно до вимог завдання 4.

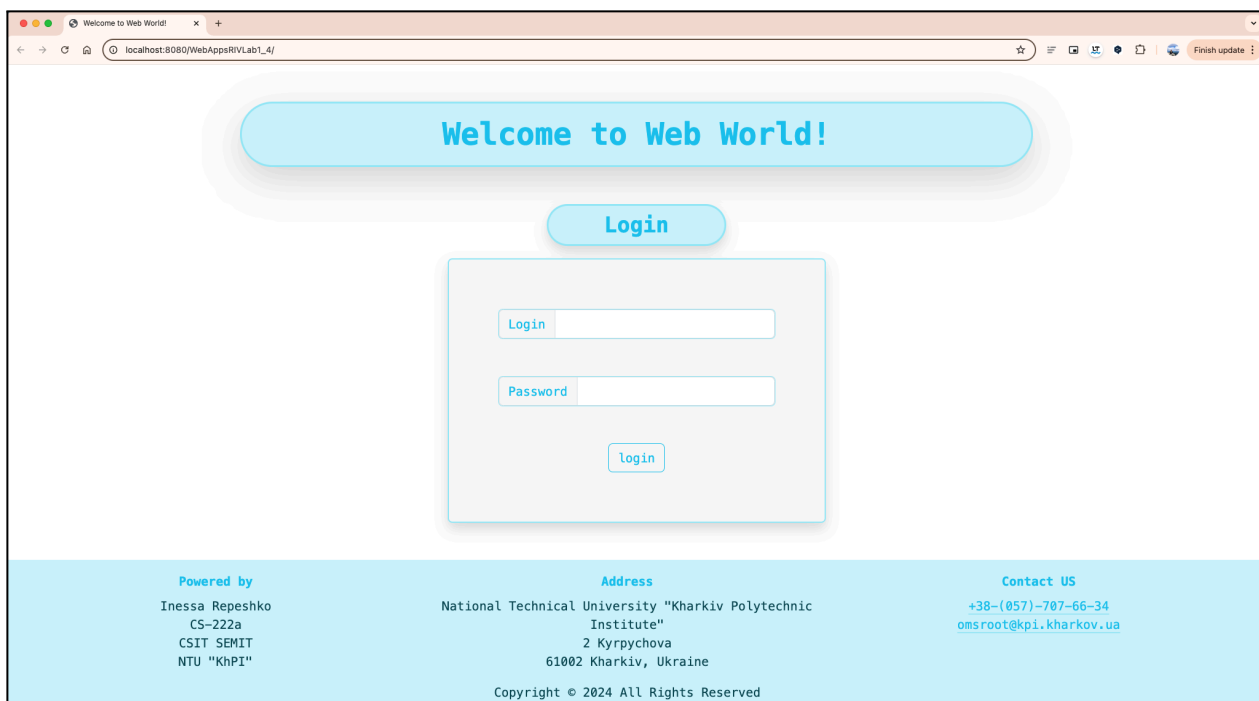


Рисунок 24 – Результат роботи вебзастосунку реалізованого до завдання 4 на стартовій сторінці

На стартовій сторінці (рис. 24) розміщена автентифікаційна форма для користувача з полями вводу “Login” та “Password”. Після відправки даних форми методом POST по натисканню кнопки “Login” відбувається авторизація з перевіркою отриманих даних у сервлеті ProcessLoginServlet.jsp.

У разі неуспішної авторизації користувачеві відображається інформативна сторінка з помилкою та кнопкою “Go Back” для повернення на стартову сторінку, що відображення на рис. 25.

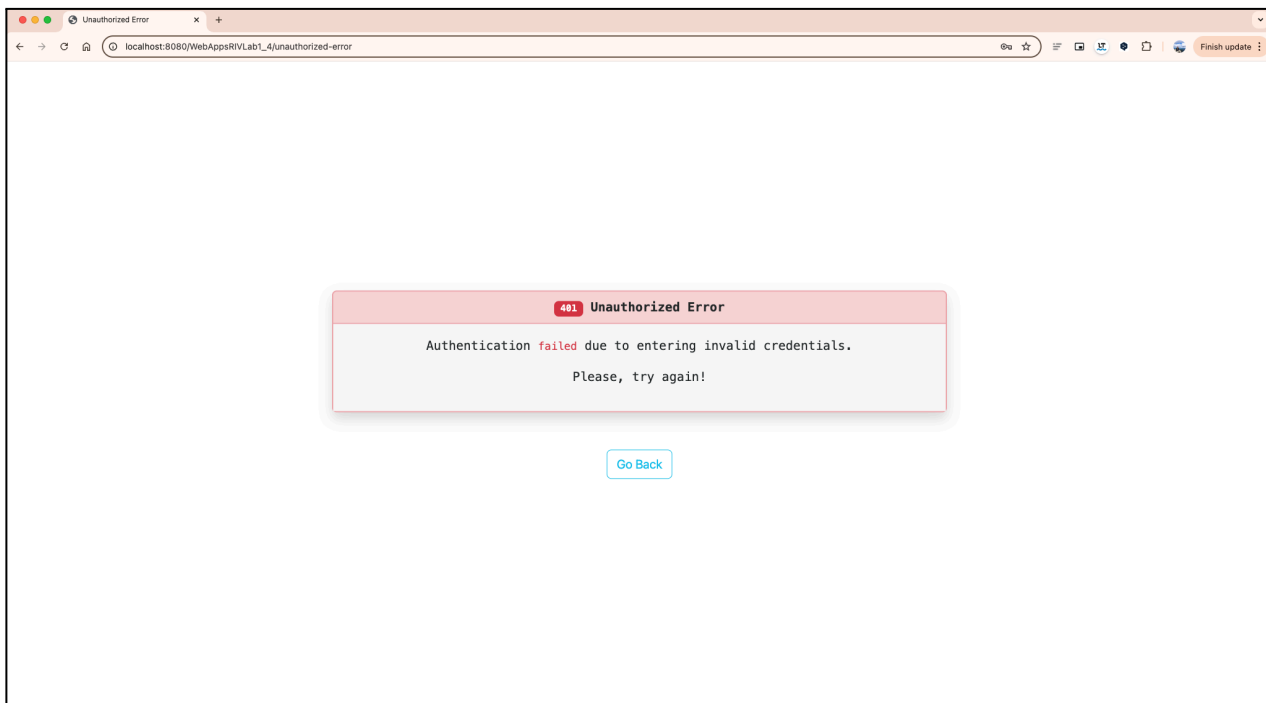


Рисунок 25 – Результат роботи вебзастосунку реалізованого до завдання 4 після неуспішної авторизації

У разі успішної авторизації користувачеві дображаться домашня сторінка вебзастосунку – JSP сторінка HelloWebWorld.jsp (рис.26). На домашній сторінці реалізованого вебзастосунку (рис. 18) є можливість переходу до сторінки з таблицею працівників деякої IT компанії по кнопці “Show” з використанням різних методів передачі запитів:

- метод `include()` класу `RequestDispatcher`;
- метод `forward()` класу `RequestDispatcher`;
- метод `sendRedirect()` класу `HttpServletResponse`.

Принципи роботи даних методів та необхідність їх застосування у даному проєкті описані у пункті 3 вище в тексті даної роботи.

Результати переходів на відповідні сторінки після натискання кнопок “Show” на домашній сторінці відображені на рис. 28-30 нижче в тексті роботи.

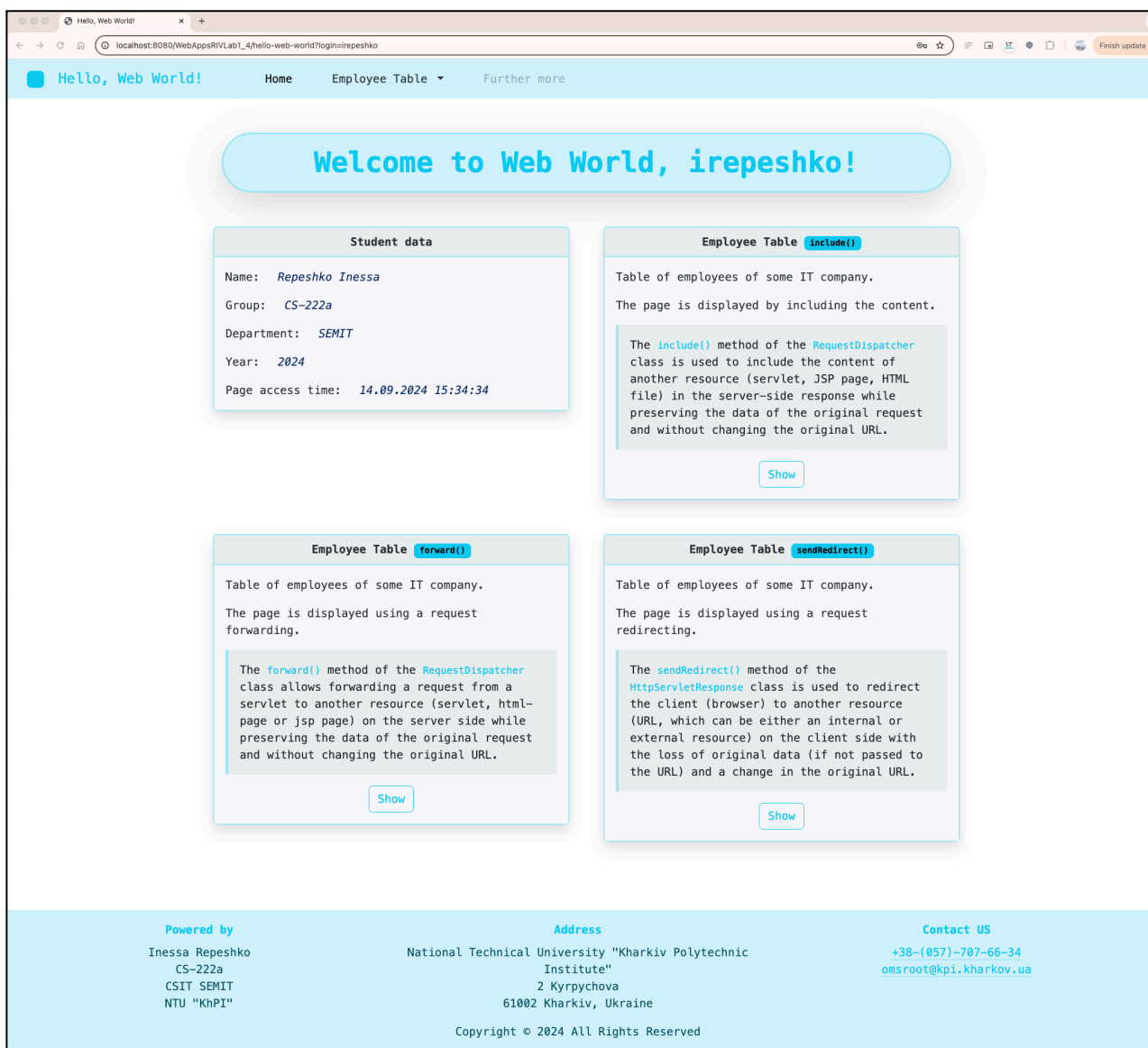


Рисунок 26 – Результат роботи вебзастосунку реалізованого до завдання 4 після успішної авторизації на домашній сторінці

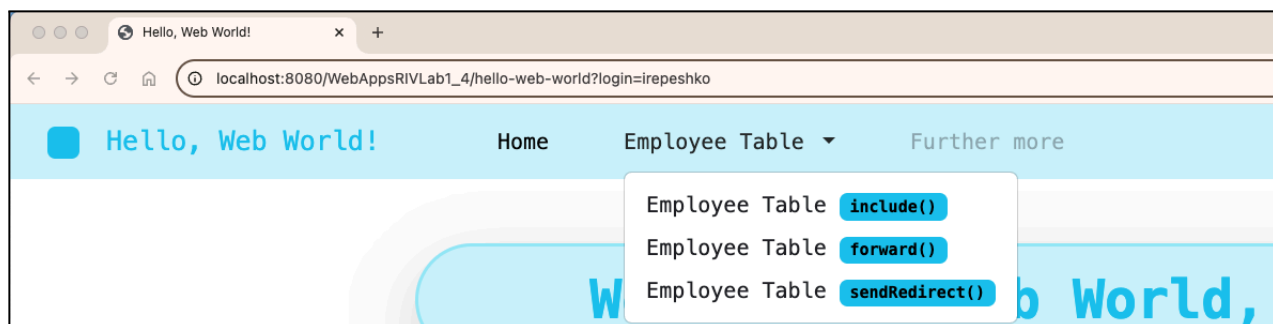


Рисунок 27 – Результат роботи вебзастосунку реалізованого до завдання 4 з відображенням меню навігаційної панелі

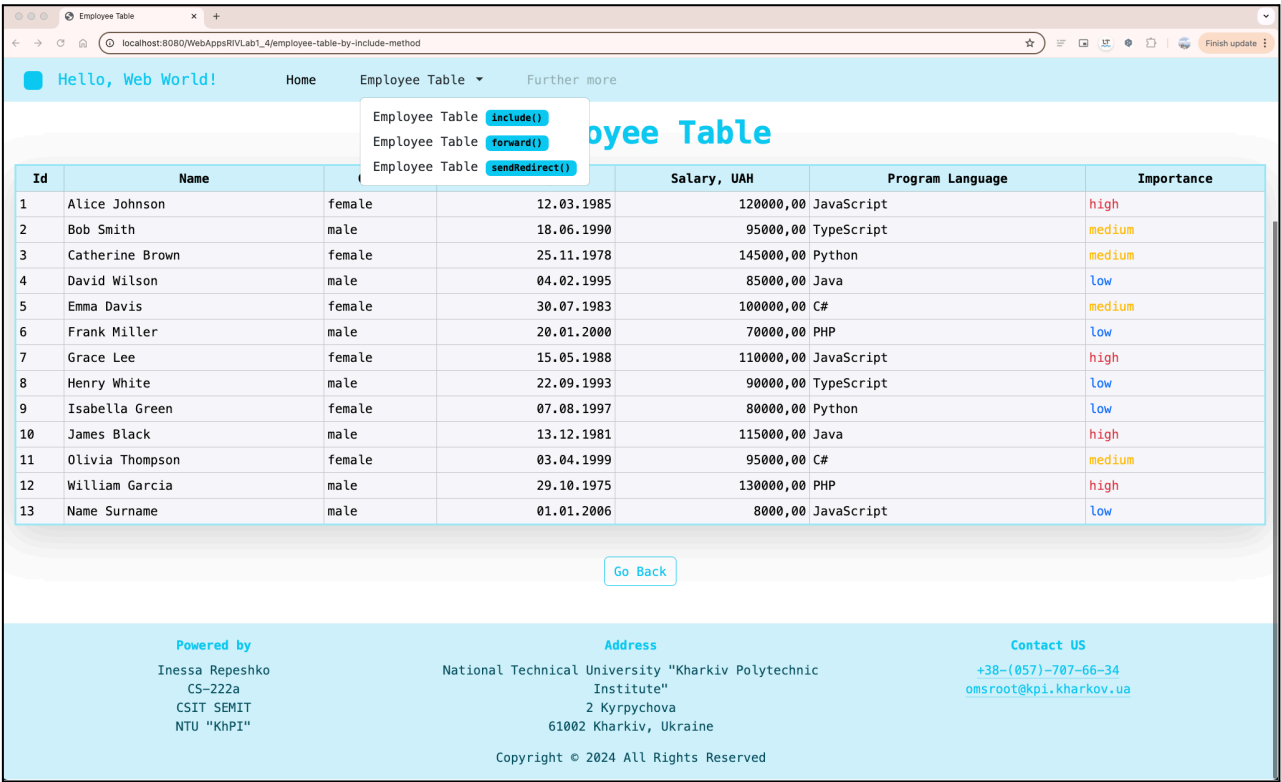


Рисунок 28 – Результат роботи вебзастосунку реалізованого до завдання 4 з відображенням сторінки з таблицею включенням методом include()

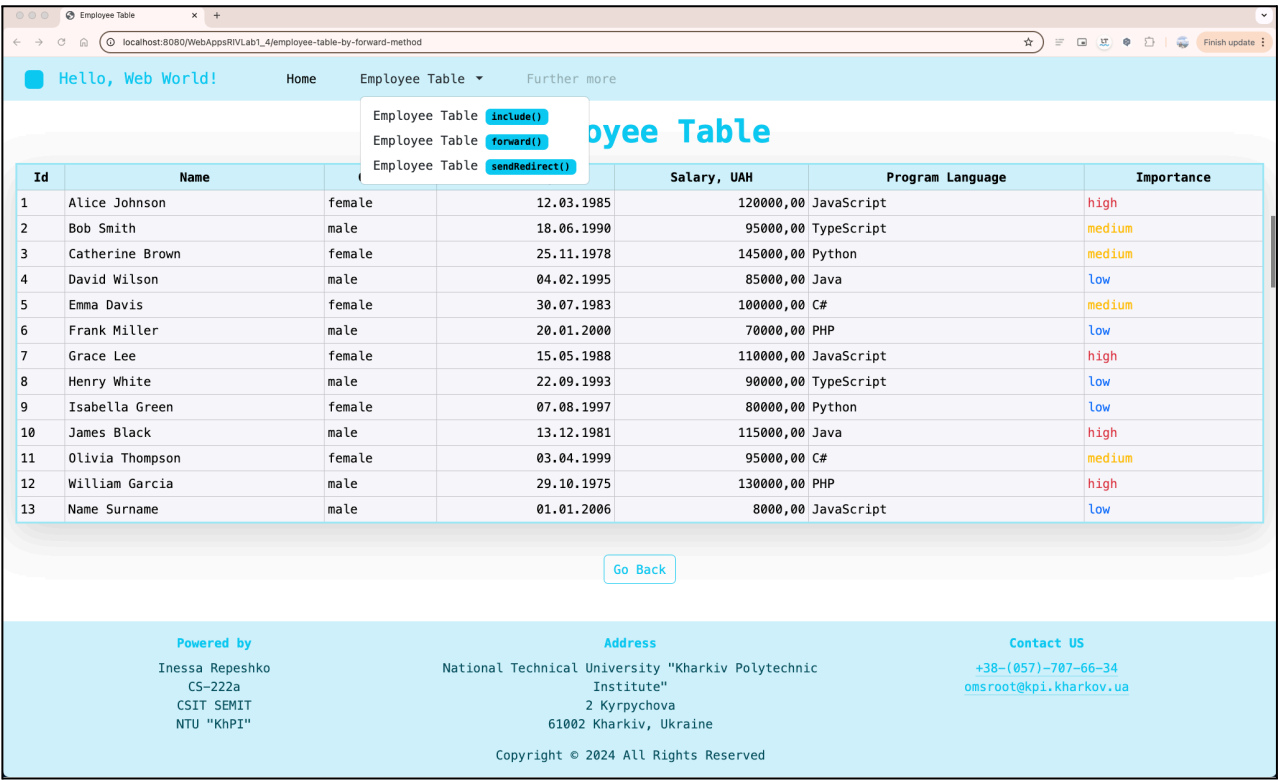


Рисунок 29 – Результат роботи вебзастосунку реалізованого до завдання 4 з відображенням сторінки з таблицею перенаправленням методом forward()

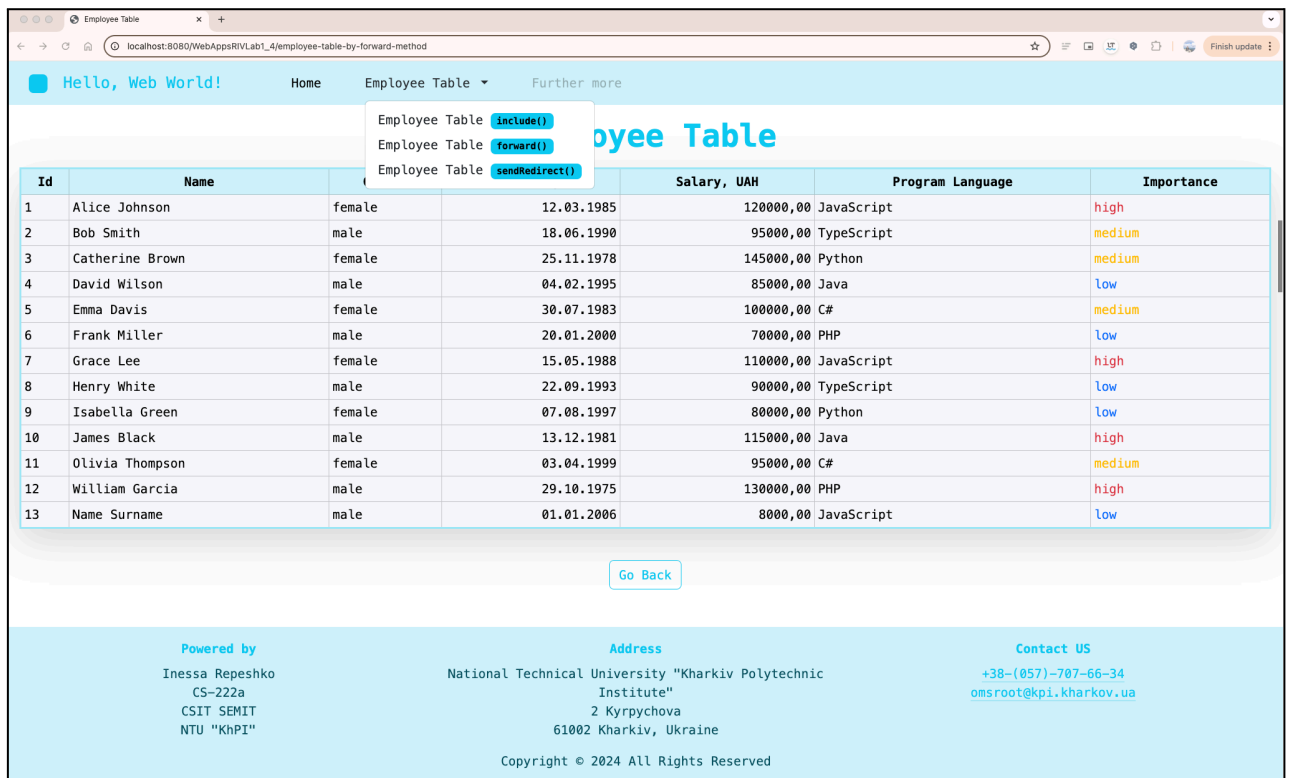


Рисунок 30 – Результат роботи вебзастосунку реалізованого до завдання 4 з відображенням сторінки з таблицею переадресацією методом `sendRedirect()`

Висновки

Отримані результати підтверджують важливість вивчення технологій Java для створення вебзастосунків. Практичні навички, набуті під час роботи з сервлетами та JSP, є критичними для фахівців у галузі програмної інженерії, особливо при розробці динамічних вебзастосунків. Важливість вивчення цих технологій полягає у здатності створювати масштабовані та надійні вебсервіси, що відповідають сучасним стандартам індустрії.

Застосування сервлетів та JSP у порівнянні з іншими технологіями, такими як Spring або JavaServer Faces (JSF), має свої переваги та недоліки. Сервлети та JSP забезпечують глибше розуміння роботи з HTTP-запитами та процесом обробки на сервері, хоча вони можуть здатися більш складними для освоєння на перших етапах. З іншого боку, більш сучасні фреймворки абстрагують багато деталей, спрощуючи розробку, але приховуючи внутрішню логіку.

Складність вивчення розглянутих технологій полягає у необхідності детального розуміння роботи вебсерверів, механізмів обробки запитів та відповіді, а також роботи з різними методами передачі даних між клієнтом та сервером. Попри це, оволодіння цими навичками є важливим етапом у підготовці кваліфікованого спеціаліста у галузі веброзробки на основі Java.

Отже, досліджені питання та отримані навички підкреслюють необхідність глибокого вивчення Java-технологій для розробки ефективних вебзастосунків. Отримані знання створюють міцний фундамент для подальшого вивчення більш складних фреймворків та підходів до розробки, що дозволяє реалізовувати масштабні та продуктивні вебрішення.

ДОДАТОК А

Код класу HelloWorldServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_1;

import java.io.*;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import jakarta.servlet.ServletException;

/**
 * The HelloWorldServlet class handles HTTP GET requests and
 * includes the content of the
 * `HelloWebWorld.jsp` page in the response.
 *
 * <p>This servlet is mapped to two URL patterns:
 * "/hello-web-world" and the root URL ("/").
 *
 * When accessed, it includes the content from the
 * `HelloWebWorld.jsp` page, allowing the
 * servlet to continue processing the response after the JSP
 * content is rendered.
 *
 * <p>The servlet uses {@link RequestDispatcher#include} to include
 * the content of the JSP page,
 * which allows the servlet to maintain control and perform further
 * processing if needed.
 *
 * @author Inessa Repeshko CS-222a
 */
@WebServlet(name = "HelloWebWorldServlet", value =
{"/hello-web-world", "/"})
public class HelloWorldServlet extends HttpServlet {
```

```

    /**
     * Handles the HTTP GET request by including the content of the
     * `HelloWebWorld.jsp` page.
     *
     * <p>This method retrieves a {@link RequestDispatcher} to
     * include the content of the
     * `HelloWebWorld.jsp` page in the response. After including
     * the JSP content, the servlet
     * can continue processing the request.
     *
     * @param request the {@link HttpServletRequest} object that
     * contains the request the client made to the servlet
     * @param response the {@link HttpServletResponse} object that
     * contains the response the servlet sends to the client
     * @throws ServletException if there is an issue with the
     * request dispatch or JSP processing
     * @throws IOException      if an input or output error occurs
     * during the handling of the request
     */
    @Override
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        response.setStatus(HttpServletResponse.SC_OK);

        RequestDispatcher dispatcher =
            request.getRequestDispatcher("/HelloWebWorld.jsp");
        dispatcher.include(request, response);
    }
}

```

ДОДАТОК Б

Код сторінки HelloWebWorld.jsp (завдання 1)

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Hello, Web World!!!</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<div class="container justify-content-center">
    <h1 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-3 my-5 bg-body-tertiary rounded">
        "Hello, Web World!!!"
    </h1>

    <div class="row align-items-start column-gap-5">
        <div class="w-50 p-0 card border border-2
border-info-subtle
            shadow mb-5 bg-body-tertiary rounded">

```

```

        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
            Student data
        </div>
        <div class="card-body bg-light font-monospace">
            <p class="card-text text-secondary-emphasis
fw-light">
                Name:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">Repeshko Inessa</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Group:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">CS-222a</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Department:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">SEMIT</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Year:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">2024</span>
            </p>
        </div>
    </div>
</div>
</body>

```

</html>

ДОДАТОК В

Код класу Employee.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.model;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.Arrays;
import java.util.stream.Collectors;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * The Employee class represents an employee with attributes such
 * as name, birthday, gender, salary,
 * programming language proficiency, and importance. It provides
 * methods to parse and validate
 * input data, ensuring the integrity of the employee's attributes.
 *
 * <p>This class also maintains a static employee number to assign
 * unique IDs to each employee
 * upon creation. The class handles various input formats for name,
 * birthday, gender, salary,
 * and programming language, applying default values when
 * necessary.
 *
 * @author Inessa Repeshko CS-222a
 */
public class Employee {
    /** Default data for creating class instances */
    private static final String DEFAULT_NAME = "Name Surname"; //
    name template
```

```

    private static final String DEFAULT_BIRTHDAY = "01.01.2006"; //
minimum working age - 18 years old

    private static final double DEFAULT_SALARY = 8000.00; //
minimum salary in Ukraine

    private static final String DEFAULT_PROGLANG = "JavaScript"; //
most popular programming language

    private static final int DEFAULT_IMPORTANCE = 0; // minimum
importance for company


    private static Long employeeNum = 0L;
    private Long id;
    private String name;
    private LocalDate birthday;
    private boolean gender;
    private double salary;
    private ProgramLanguages programLanguage;
    private int importance;


    /**
     * Constructs a new Employee instance, validating and setting
the attributes.
     *
     * @param id           the id of the employee as a long
number.
     *
     *                    If is null, the value employeeNum is
used.
     * @param name        the name of the employee, formatted
as DEFAULT_NAME.
     *
     *                    If the name is invalid or too short,
a value DEFAULT_NAME is used.
     * @param birthday    the employee's birthdate in the
format "dd.MM.yyyy".
     *
     *                    If the format is invalid, a date of
DEFAULT_BIRTHDAY is used.
     * @param gender      the gender of the employee, which
must be either true (male) or false (female).

```

```

    * @param salary          the employee's salary, which must be
a positive double number.
    *
    *                      If the salary is invalid, a value of
DEFAULT_SALARY is used.
    * @param programLanguage the employee's primary programming
language as a string.
    *
    *                      This value is matched with the
ProgramLanguages enum.
    * @param importance      the employee's importance for company
as 0 (low), 1 (medium) or 2 (high).
    *
    *                      If the importance is invalid, a value
of DEFAULT_IMPORTANCE is used.
    */
    public Employee(
        long id,
        String name,
        String birthday,
        boolean gender,
        double salary,
        String programLanguage,
        int importance
    ) {
        employeeNum++;
        this.id = (id != -1) ? id : employeeNum;
        this.name = parseNameOrDefault(name);
        this.birthday = parseBirthdayOrDefault(birthday);
        this.gender = gender;
        this.salary = parseSalaryOrDefault(salary);
        this.programLanguage =
parseProgLangOrDefault(programLanguage);
        this.importance = parseImportanceOrDefault(importance);
    }

    /**
    * Parses the employee's name and ensures it is correctly
formatted.

```

```

*
* @param name the name of the employee
* @return the formatted name with the first letter of each
word capitalized, or
* DEFAULT_NAME if the input name is invalid.
*/
private String parseNameOrDefault(String name) {
    name = name.length() > 5 ? name : DEFAULT_NAME;

    return Arrays.stream(name.split("\\s+"))
        .map(word -> word.substring(0, 1).toUpperCase() +
word.substring(1).toLowerCase())
        .collect(Collectors.joining(" "));
}

/**
* Parses the employee's birthday from a string and validates
its format.
*
* @param birthday the birthdate string in the format
"dd.MM.yyyy"
* @return the parsed LocalDate object, or a default date of
"01.01.2006"
* (18 years is a minimum working age in Ukraine) if parsing
fails.
*/
private LocalDate parseBirthdayOrDefault(String birthday) {
    DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd.MM.yyyy");

    try {
        return LocalDate.parse(birthday, formatter);
    } catch (DateTimeParseException e) {
        return LocalDate.parse(DEFAULT_BIRTHDAY, formatter);
    }
}

```

```

/**
 * Validates the employee's salary.
 *
 * @param salary the salary amount
 * @return the salary if it is greater than 0, or
DEFAULT_SALARY (minimum salary in Ukraine) as the default.
 */
private double parseSalaryOrDefault(double salary) {
    return salary > 0 ? salary : DEFAULT_SALARY;
}

/**
 * Parses the given programming language string and returns the
corresponding ProgramLanguages enum value.
 *
 * <p>If the provided program language does not match any enum
values, the method defaults to the first enum
 * value in the ProgramLanguages array (usually the default
value). The method uses the
 * {@link ProgramLanguages#getEnumIndex(String)} to find the
appropriate index based on the display name of the language.
 *
 * @param programLanguage the programming language as a string
 * @return the corresponding ProgramLanguages enum value, or
the first enum value if the input is invalid.
 */
private ProgramLanguages parseProgLangOrDefault(String
programLanguage) {
    int index = ProgramLanguages.getEnumIndex(programLanguage);
    index = index > -1 ? index : 0;

    return ProgramLanguages.values()[index];
}

/**

```

```

    * Parses the importance level of the employee and returns a
valid default value.
    *
    * @param importance the importance level as an integer
    * @return the validated importance level, either 0 (low), 1
(medium), 2 (high), or DEFAULT_IMPORTANCE 0 (low).
    */
    private int parseImportanceOrDefault(int importance) {
        return (importance == 0 || importance == 1 || importance ==
2)

            ? importance
            : DEFAULT_IMPORTANCE;
    }

    /**
     * Retrieves the employee's id.
     *
     * @return the employee's id.
     */
    public Long getId() {
        return id;
    }

    /**
     * Retrieves the employee's name.
     *
     * @return the employee's name as a string.
     */
    public String getName() {
        return name;
    }

    /**
     * Retrieves the employee's gender.
     *
     * @return the employee's gender as a boolean.

```

```

    */
    public boolean getGender() {
        return gender;
    }

    /**
     * Retrieves the employee's gender.
     *
     * @return the employee's gender as a string.
     */
    public String getGenderAsString() {
        return getGender() ? "male" : "female";
    }

    /**
     * Retrieves the employee's birthday as a string in the format
     "dd.MM.yyyy".
     *
     * @return the formatted birthday string.
     */
    public String getBirthdayAsString() {
        return
birthday.format(DateTimeFormatter.ofPattern("dd.MM.yyyy"));
    }

    /**
     * Retrieves the employee's salary formatted as currency.
     *
     * @return the salary formatted as a string with two decimal
places and "UAH".
     */
    public String getSalary() {
        return String.format("%.2f", salary);
    }

```

```
/**
 * Retrieves the employee's primary programming language.
 *
 * @return the programming language as a string.
 */
public String getProgramLanguage() {
    return programLanguage.getDisplayName();
}

/**
 * Retrieves the importance level of the employee.
 *
 * @return the importance level as an integer.
 */
public int getImportance() {
    return importance;
}

/**
 * Retrieves the importance level of the employee.
 *
 * @return the importance level as a string.
 */
public String getImportanceAsString() {
    switch (getImportance()) {
        case 1:
            return "medium";
        case 2:
            return "high";
        default:
            return "low";
    }
}

/**
```



```

    * Constructs a string representation of the employee object in
the form of URL parameters.
    *
    * <p>This method returns a string that contains key-value
pairs of the employee's attributes,
    * formatted as if they were passed as URL query parameters.
The string includes the employee's
    * ID, name, birthday, gender, salary, programming language,
and importance level.
    *
    * @return a string representing the employee's attributes
formatted as URL parameters.
    */
    public String getEmployeeAsParameters() {
        StringBuilder parameters = new StringBuilder();

        parameters.append("Employee {");
        parameters.append("id=").append(getId());
        parameters.append(", name=").append(getName());
        parameters.append(",
birthday=").append(getBirthdayAsString());
        parameters.append(", gender=").append(getGender());
        parameters.append(", salary=").append(getSalary());
        parameters.append(",
programLanguage=").append(getProgramLanguage());
        parameters.append(", importance=").append(getImportance());
        parameters.append(" }");

        return parameters.toString();
    }

    /**
    * Parses a string representation in the form of URL parameters
to an Employee object.
    *
    * <p>This method returns an Employee object with its data

```

```

    * (id, name, birthday, gender, salary, programming language,
    and importance level)
    * parsed from string that contains key-value pairs of the
    employee's attributes,
    * formatted as if they were passed as URL query parameters.
    *
    * @return an Employee object parsed from the employee's
    attributes formatted as URL parameters.
    */
    public static Employee parseEmployeeFromParameters(String
parameters) {
        Matcher idMatcher =
Pattern.compile("id=(\\d+)").matcher(parameters),
        nameMatcher =
Pattern.compile("name=' ([a-zA-Z]+) (\\s+[a-zA-Z]+) *'").matcher(para
meters),
        birthdayMatcher =
Pattern.compile("birthday=(\\d{2}.\\d{2}.\\d{4})").matcher(paramet
ers),
        genderMatcher =
Pattern.compile("gender=(true|false)").matcher(parameters),
        salaryMatcher =
Pattern.compile("salary=(\\d+\\.\\d+)").matcher(parameters),
        programLanguageMatcher =
Pattern.compile("programLanguage= ([a-zA-Z]+) (\\s+[a-zA-Z]+) *").mat
cher(parameters),
        importanceMatcher =
Pattern.compile("importance=(\\d+)").matcher(parameters);

        long id = idMatcher.find()
            ? Long.parseLong(idMatcher.group(1))
            : -1;

        String name = nameMatcher.find()
            ? nameMatcher.group(1)
            : DEFAULT_NAME;

        String birthday = birthdayMatcher.find()

```

```
        ? birthdayMatcher.group(1)
        : DEFAULT_BIRTHDAY;

    boolean gender = genderMatcher.find()
        ? Boolean.parseBoolean(genderMatcher.group(1))
        : false;

    double salary = salaryMatcher.find()
        ? Double.parseDouble(salaryMatcher.group(1))
        : DEFAULT_SALARY;

    String programLanguage = programLanguageMatcher.find()
        ? programLanguageMatcher.group(1)
        : DEFAULT_PROGLANG;

    int importance = importanceMatcher.find()
        ? Integer.parseInt(importanceMatcher.group(1))
        : DEFAULT_IMPORTANCE;

    return new Employee(id, name, birthday, gender, salary,
programLanguage, importance);
}
}
```

ДОДАТОК Г

Код класу EmployeeList.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.model;

import java.util.ArrayList;
import java.util.List;

/**
 * The EmployeeList class is a singleton that extends the ArrayList
 * of Employee objects. It contains
 * a static method to retrieve the single instance of the class and
 * initializes the list with
 * predefined employee data when first accessed. This class ensures
 * that only one instance of
 * the employee list exists throughout the application.
 *
 * <p>It provides a default set of employees with various
 * attributes such as name, birthday, gender,
 * salary, program language, and importance. This list can be
 * accessed and modified by the user.
 *
 * @author Inessa Repeshko CS-222a
 */
public class EmployeeList extends ArrayList<Employee> {
    private static final long serialVersionUID = 1L;
    private static EmployeeList instance;

    /**
     * Private constructor to prevent instantiation from outside
     the class.
     * This ensures that the EmployeeList follows the singleton
     design pattern.
     */
}
```

```

private EmployeeList() {
}

/**
 * Provides access to the single instance of EmployeeList. If
the instance does not
 * exist, it is created and populated with a predefined list of
employees.
 *
 * <p>This method initializes the employee list with various
employee records, including
 * different attributes such as name, birthday, gender, salary,
program language,
 * and importance. If an instance already exists, it simply
returns the existing one.
 *
 * @return the singleton instance of EmployeeList.
 */
public static EmployeeList getInstance() {
    if (instance == null) {
        instance = new EmployeeList();

        instance.add(new Employee("Alice Johnson",
"12.03.1985", false, 120000, "JavaScript", 2));
        instance.add(new Employee("Bob Smith", "18.06.1990",
true, 95000, "TypeScript", 1));
        instance.add(new Employee("Catherine Brown",
"25.11.1978", false, 145000, "Python", 1));
        instance.add(new Employee("David Wilson", "04.02.1995",
true, 85000, "Java", 0));
        instance.add(new Employee("Emma Davis", "30.07.1983",
false, 100000, "C#", 1));
        instance.add(new Employee("Frank Miller", "20.01.2000",
true, 70000, "PHP", 0));
        instance.add(new Employee("Grace Lee", "15.05.1988",
false, 110000, "JavaScript", 2));
    }
}

```

```

        instance.add(new Employee("Henry White", "22.09.1993",
true, 90000, "TypeScript", 0));
        instance.add(new Employee("Isabella Green",
"07.08.1997", false, 80000, "Python", 0));
        instance.add(new Employee("James Black", "13.12.1981",
true, 115000, "Java", 2));
        instance.add(new Employee("Olivia Thompson",
"03.04.1999", false, 95000, "C#", 1));
        instance.add(new Employee("william garcia",
"29.10.1975", true, 130000, "PHP", 2));
        instance.add(new Employee("test", "birthday", true, -1,
"proglang", -1));
    }

    return instance;
}

/**
 * Retrieves a list of all employees currently stored in the
EmployeeList.
 *
 * <p>This method returns a new copy of the current
EmployeeList to prevent
 * external modifications to the original list. The returned
list contains
 * all employee objects that have been added to the
EmployeeList.
 *
 * @return a new ArrayList containing all employees in the
EmployeeList.
 */
public List<Employee> getEmployees() {
    return new ArrayList<>(this);
}
}

```


ДОДАТОК Д

Код класу ProgramLanguages.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.model;

/**
 * The ProgramLanguages enum represents a set of common programming
 * languages, each associated with
 * a display name. This enum provides methods to retrieve the
 * display name of each language and
 * determine the index of a language based on its display name.
 *
 * <p>Each constant in the enum corresponds to a programming
 * language, and the display name is the
 * human-readable name of the language.
 *
 * @author Inessa Repeshko CS-222a
 */
public enum ProgramLanguages {
    javascript("JavaScript"),
    typescript("TypeScript"),
    python("Python"),
    java("Java"),
    csharp("C#"),
    php("PHP");

    private final String displayName;

    /**
     * Constructor for the ProgramLanguages enum.
     *
     * @param displayName the human-readable name of the
     * programming language.
     */
    ProgramLanguages(String displayName) {
```



```

        this.displayName = displayName;
    }

    /**
     * Retrieves the display name of the programming language.
     *
     * @return the display name of the language as a string.
     */
    public String getDisplayName() {
        return displayName;
    }

    /**
     * Finds the index of the programming language in the enum
    based on its display name.
     *
     * <p>If the given value matches the display name of a
    programming language, the method
     * returns its index in the enum; otherwise, it returns -1 if
    no match is found.
     *
     * @param value the display name of the programming language to
    search for.
     * @return the index of the programming language in the enum,
    or -1 if not found.
     */
    public static int getEnumIndex(String value) {
        for (int i = 0; i < values().length; i++) {
            if (values()[i].getDisplayName().equals(value)) {
                return i;
            }
        }

        return -1;
    }
}

```

ДОДАТОК Е

Код класу EmployeeTableIncludeServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.servlets;

import java.io.*;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
import ntukhpi.csit.semit.riv.webappsrivlab1_2.model.*;
import java.util.List;

/**
 * The EmployeeTableServlet is responsible for processing HTTP GET
 * requests and including the
 * content of the `EmployeeTable.jsp` page in the response.
 *
 * <p>This servlet is mapped to the URL pattern "/employee-table"
 * and serves the purpose of rendering
 * an HTML table containing employee data. The servlet includes the
 * content of the JSP page,
 * allowing for further response processing after the JSP content
 * is rendered.
 *
 * <p>The `EmployeeTable.jsp` page displays the employee data in a
 * table format, while this servlet
 * ensures that the correct data is passed to the page for display.
 *
 * <p>Inclusion of the JSP page's content is done via a {@link
 * RequestDispatcher#include}, ensuring that
 * control is returned to the servlet after the page is processed,
 * allowing for additional logic
```

```

* or response handling.
*
* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "EmployeeTableIncludeServlet", value =
"/employee-table-by-include-method")
public class EmployeeTableIncludeServlet extends HttpServlet {

    /**
     * Processes the HTTP GET request by including the content of
the `EmployeeTable.jsp` page.
     *
     * <p>This method uses a {@link RequestDispatcher} to include
the content of the JSP page
     * in the response, which displays an HTML table of employees.
The inclusion allows the
     * servlet to continue processing after rendering the JSP
content.
     *
     * <p>The JSP page is responsible for rendering the employee
table using the data provided
     * in the request, while the servlet can continue to process or
modify the response after
     * the inclusion.
     *
     * @param request the {@link HttpServletRequest} object that
contains the client's request
     * @param response the {@link HttpServletResponse} object that
contains the servlet's response to the client
     * @throws ServletException if the request for the inclusion
cannot be handled
     * @throws IOException      if an input or output error occurs
while the servlet is handling the request
     */
    @Override

```

```
protected void doGet(HttpServletRequest request,
HttpServletRequest response) throws ServletException, IOException
{
    EmployeeList employeeList = EmployeeList.getInstance();
    List<Employee> employees = employeeList.getEmployees();

    request.setAttribute("employees", employees);
    response.setStatus (HttpServletResponse.SC_OK);

    RequestDispatcher dispatcher =
request.getRequestDispatcher("/EmployeeTable.jsp");
    dispatcher.include(request, response);
}
}
```

ДОДАТОК Ж

Код сторінки HelloWebWorld.jsp (завдання 2)

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Hello, Web World!!!</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<div class="container justify-content-center">
    <h1 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-3 my-5 bg-body-tertiary rounded">
        Hello, Web World!!!
    </h1>

    <div class="row align-items-start column-gap-5">
        <div class="col w-50 p-0 card border border-2
border-info-subtle
            shadow mb-5 bg-body-tertiary rounded">

```

```

        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
            Student data
        </div>
        <div class="card-body bg-light font-monospace">
            <p class="card-text text-secondary-emphasis
fw-light">
                Name:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">Repeshko Inessa</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Group:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">CS-222a</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Department:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">SEMIT</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Year:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">2024</span>
            </p>
        </div>
    </div>

    <div class="col w-50 p-0 card border border-2
border-info-subtle

```

```

        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
            Employee Table
            <span class="badge text-bg-info">include()</span>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table of employees of some IT company.</p>
                <p>The page is displayed by including the
content.</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                The <code class="text-info">include()</code>
method of the
                <code
class="text-info">RequestDispatcher</code> class is used to
include
                the content of another resource (servlet, JSP
page, HTML file) in the server-side response
                while preserving the data of the original
request and without changing the original URL.
            </div>

            <div class="d-flex justify-content-center">
                <a href="employee-table-by-include-method"
class="btn btn-outline-info" role="button">Show</a>
            </div>
        </div>
    </div>

```

```
    </div>  
</div>  
</body>  
</html>
```


ДОДАТОК И

Код сторінки EmployeeTable.jsp (завдання 2)

```

<%@ page import="java.util.List" %>
<%@ page import="java.util.ArrayList" %>
<%@ page import="ntukhpi.csit.semit.riv.webappsrivlab1_2.model.*"
%>
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Employee Table</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body class="mx-3 my-3 font-monospace">
<h1 class="text-info fs-1 mb-3 fw-semibold">
    Employee Table
</h1>
<table class="table table-sm table-hover font-monospace
        table-bordered border border-3 border-info-subtle
shadow-lg">
    <thead class="table-info text-center fw-medium
text-secondary-emphasis">

```

```

        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Gender</th>
            <th>Birthday</th>
            <th>Salary, UAH</th>
            <th>Program Language</th>
            <th>Importance</th>
        </tr>
    </thead>
    <tbody class="table-light fw-light text-primary-emphasis">
        <%
            List<Employee> employeesToDisplay = new ArrayList<>();
            employeesToDisplay = (List<Employee>)
request.getAttribute("employees");

            for (Employee employee: employeesToDisplay) {
                String textColor;

                switch (employee.getImportance()) {
                    case 1:
                        textColor = "text-warning";
                        break;
                    case 2:
                        textColor = "text-danger";
                        break;
                    default:
                        textColor = "text-primary";
                };

                out.println("<tr>");
                out.println("<td class=\"text-start\">" +
employee.getId() + "</td>");
                out.println("<td class=\"text-start\">" +
employee.getName() + "</td>");

```

```

        out.println("<td class=\"text-start\">" +
employee.getGenderAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getBirthdayAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getSalary() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getProgramLanguage()+ "</td>");
        out.println("<td class=\"text-start \" + textColor +
\"\">" + employee.getImportanceAsString() + "</td>");
        out.println("</tr>");
    }
    %>
</tbody>
</table>
<br>
<div class="d-flex justify-content-center">
    <a href="/WebAppsRIVLab1_2/hello-web-world" class="btn
btn-outline-info" role="button">Go Back</a>
</div>
</body>
</html>

```

ДОДАТОК К

Код класу EmployeeTableForwardServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.servlets;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import ntukhpi.csit.semit.riv.webappsrivlab1_2.model.Employee;
import ntukhpi.csit.semit.riv.webappsrivlab1_2.model.EmployeeList;

import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;

/**
 * The EmployeeTableForwardServlet handles HTTP GET requests and
 * forwards the request to the
 * `EmployeeTable.jsp` page, passing the list of employees as a
 * request attribute.
 *
 * <p>This servlet is mapped to the URL pattern
 * "/employee-table-by-forward-method" and retrieves the
 * employee list from the {@link EmployeeList} singleton. It
 * forwards the list to the JSP page
 * for rendering in a table format.
 *
 * <p>The servlet uses {@link RequestDispatcher#forward} to forward
 * the request and response
 * to the JSP page, which will handle the presentation of employee
 * data.
```

```

*
* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "EmployeeTableForwardServlet", value =
"/employee-table-by-forward-method")
public class EmployeeTableForwardServlet extends HttpServlet {

    /**
     * Handles the HTTP GET request by retrieving the list of
employees and forwarding it
     * to the `EmployeeTable.jsp` page for rendering.
     *
     * <p>This method obtains the employee list from the {@link
EmployeeList} singleton, sets
     * it as a request attribute, and forwards the request to the
JSP page for display.
     * The content type of the response is set to "text/html" with
UTF-8 encoding to support
     * proper rendering of the employee data.
     *
     * @param request the {@link HttpServletRequest} object
containing the client's request
     * @param response the {@link HttpServletResponse} object
containing the servlet's response
     * @throws ServletException if there is an issue with the
request dispatch or JSP processing
     * @throws IOException      if an input or output error occurs
while handling the request
     */
    @Override
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
    {
        EmployeeList employeeList = EmployeeList.getInstance();
        List<Employee> employees = employeeList.getEmployees();

```

```
        request.setAttribute("employees", employees);  
        response.setStatus(HttpServletResponse.SC_OK);  
  
        RequestDispatcher dispatcher =  
request.getRequestDispatcher("/EmployeeTable.jsp");  
        dispatcher.forward(request, response);  
    }  
}
```

ДОДАТОК Л

Код класу EmployeeTableSendRedirectServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_2.servlets;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import ntukhpi.csit.semit.riv.webappsrivlab1_2.model.Employee;
import ntukhpi.csit.semit.riv.webappsrivlab1_2.model.EmployeeList;

import java.io.IOException;
import java.util.List;

/**
 * The EmployeeTableSendRedirectServlet class handles HTTP GET
 * requests by redirecting the client
 * to the `EmployeeTable.jsp` page with a list of employee IDs as
 * URL parameters.
 *
 * <p>This servlet is mapped to the URL pattern
 * "/employee-table-by-sendredirect-method" and retrieves
 * the employee list from the {@link EmployeeList} singleton. It
 * builds a query string containing
 * the employee IDs and uses the {@link
 * HttpServletResponse#sendRedirect} method to redirect
 * the client to the JSP page.
 *
 * <p>Unlike forwarding, the sendRedirect method sends a new
 * request to the server, meaning the client
 * is aware of the redirection, and the URL in the browser changes
 * to the new target.
 *
```

```

* @author Inessa Repeshko CS-222a
*/
@WebServlet(name = "EmployeeTableSendRedirectServlet", value =
"/employee-table-by-sendredirect-method")
public class EmployeeTableSendRedirectServlet extends HttpServlet
{

    /**
     * Handles the HTTP GET request by retrieving the list of
employees and redirecting the client
     * to the `EmployeeTable.jsp` page with employee IDs passed as
URL parameters.
     *
     * <p>This method builds a comma-separated list of employee IDs
and appends it to the URL
     * as a query string. It then redirects the client to the JSP
page for further processing.
     *
     * @param request the {@link HttpServletRequest} object
containing the client's request
     * @param response the {@link HttpServletResponse} object
containing the servlet's response
     * @throws IOException if an input or output error occurs while
handling the request
     */
    @Override
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws IOException {
        EmployeeList employeeList = EmployeeList.getInstance();
        List<Employee> employees = employeeList.getEmployees();

        StringBuilder employeeIds = new StringBuilder();

        for (Employee employee : employees) {
            employeeIds.append(employee.getId()).append(",");
        }
    }
}

```



```
if (employeeIds.length() > 0) {  
    employeeIds.setLength(employeeIds.length() - 1);  
}  
  
response.setStatus(HttpServletResponse.SC_FOUND);  
response.sendRedirect(request.getContextPath() +  
"/EmployeeTable.jsp?employeeIds=" + employeeIds.toString());  
}  
}
```

ДОДАТОК М

Код сторінки HelloWebWorld.jsp (завдання 3)

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Hello, Web World!!!</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<div class="container justify-content-center">
    <h1 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-3 my-5 bg-body-tertiary rounded">
        Hello, Web World!!!
    </h1>

    <div class="row align-items-start column-gap-5">
        <div class="col w-50 p-0 card border border-2
border-info-subtle

```

```

        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
        Student data
    </div>
    <div class="card-body bg-light font-monospace">
        <p class="card-text text-secondary-emphasis
fw-light">
            Name:
            <span class="text-primary-emphasis fw-medium
fst-italic ms-3">Repeshko I.V.</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Group:
            <span class="text-primary-emphasis fw-medium
fst-italic ms-3">CS-222a</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Department:
            <span class="text-primary-emphasis fw-medium
fst-italic ms-3">SEMIT</span>
        </p>
        <p class="card-text text-secondary-emphasis
fw-light">
            Year:
            <span class="text-primary-emphasis fw-medium
fst-italic ms-3">2024</span>
        </p>
    </div>
</div>

```

```

        <div class="col w-50 p-0 card border border-2
border-info-subtle
        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
            Employee Table
            <span class="badge text-bg-info">include()</span>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table of employees of some IT company.</p>
                <p>The page is displayed by including the
content.</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                The <code class="text-info">include()</code>
method of the
                <code
class="text-info">RequestDispatcher</code> class is used to
include
                the content of another resource (servlet, JSP
page, HTML file) in the server-side response
                while preserving the data of the original
request and without changing the original URL.
            </div>

            <div class="d-flex justify-content-center">
                <a href="employee-table-by-include-method"
class="btn btn-outline-info" role="button">Show</a>
            </div>

```

```

        </div>
    </div>
</div>

<div class="row align-items-start column-gap-5">
    <div class="col w-50 p-0 card border border-2
border-info-subtle
        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">
            Employee Table
            <span class="badge text-bg-info">forward()</span>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table of employees of some IT company.</p>
                <p>The page is displayed using a request
forwarding.</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                The <code class="text-info">forward()</code>
method of the
                <code
class="text-info">RequestDispatcher</code> class allows forwarding
                a request from a servlet to another resource
(servlet, html-page or jsp page) on the server side
                while preserving the data of the original
request and without changing the original URL.
            </div>

```

```

        <div class="d-flex justify-content-center">
            <a href="employee-table-by-forward-method"
class="btn btn-outline-info" role="button">Show</a>
        </div>
    </div>
</div>

    <div class="col w-50 p-0 card border border-2
border-info-subtle
        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">
            Employee Table
            <span class="badge
text-bg-info">sendRedirect()</span>
        </div>

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light">
                <p>Table of employees of some IT company.</p>
                <p>The page is displayed using a request
redirecting.</p>
            </div>

            <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
                The <code
class="text-info">sendRedirect()</code> method of the
                <code
class="text-info">HttpServletResponse</code> class is used to
redirect the client (browser)
                to another resource (URL, which can be either
an internal or external resource) on the client side

```

```
        with the loss of original data (if not passed
to the URL) and a change in the original URL.
    </div>

    <div class="d-flex justify-content-center">
        <a href="employee-table-by-sendredirect-method"
class="btn btn-outline-info" role="button">Show</a>
    </div>
</div>
</div>
</div>
</div>
</body>
</html>
```

ДОДАТОК Н

Код сторінки EmployeeTable.jsp (завдання 3)

```

<%@ page import="java.util.List" %>
<%@ page import="java.util.ArrayList" %>
<%@ page import="ntukhpi.csit.semit.riv.webappsrivlab1_2.model.*"
%>
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Employee Table</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body class="mx-3 my-3 font-monospace">
<h1 class="text-info fs-1 mb-3 fw-semibold">
    Employee Table
</h1>
<table class="table table-sm table-hover font-monospace
        table-bordered border border-3 border-info-subtle
shadow-lg">
    <thead class="table-info text-center fw-medium
text-secondary-emphasis">

```



```

        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Gender</th>
            <th>Birthday</th>
            <th>Salary, UAH</th>
            <th>Program Language</th>
            <th>Importance</th>
        </tr>
    </thead>
    <tbody class="table-light fw-light text-primary-emphasis">
        <%
            String employeeIdsParam =
request.getParameter("employeeIds");

            List<Long> employeeIds = new ArrayList<>();
            List<Employee> employeesToDisplay = new ArrayList<>();

            if (employeeIdsParam != null &&
!employeeIdsParam.isEmpty()) {
                String[] ids = employeeIdsParam.split(",");

                for (String id : ids) {
                    employeeIds.add(Long.parseLong(id));
                }

                EmployeeList employeeList =
EmployeeList.getInstance();

                for (Employee employee :
employeeList.getEmployees()) {
                    if (employeeIds.contains(employee.getId())) {
                        employeesToDisplay.add(employee);
                    }
                }
            } else {

```

```

        employeesToDisplay = (List<Employee>)
request.getAttribute("employees");
    }

    for (Employee employee: employeesToDisplay) {
        String textColor;

        switch (employee.getImportance()) {
            case 1:
                textColor = "text-warning";
                break;
            case 2:
                textColor = "text-danger";
                break;
            default:
                textColor = "text-primary";
        };

        out.println("<tr>");
        out.println("<td class=\"text-start\">" +
employee.getId() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getName() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getGenderAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getBirthdayAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getSalary() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getProgramLanguage()+ "</td>");
        out.println("<td class=\"text-start \" + textColor +
\"\">" + employee.getImportanceAsString() + "</td>");
        out.println("</tr>");
    }
%>

```

```
    </tbody>
</table>
<br>
<div class="d-flex justify-content-center">
  <a href="/WebAppsRIVLab1_2/hello-web-world" class="btn
btn-outline-info" role="button">Go Back</a>
</div>
</body>
</html>
```

ДОДАТОК II

Код класу ProcessLoginServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_4.servlets;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * The ProcessLoginServlet class handles login requests by
 * validating the user's credentials
 * and redirecting to different pages based on the authentication
 * result.
 *
 * <p>This servlet is mapped to the URL pattern "/process-login"
 * and handles both GET and POST requests.
 * Upon receiving a login request, it compares the provided login
 * and password with predefined values.
 * If the credentials are valid, the user is redirected to the
 * welcome page; otherwise, they are
 * redirected to an error page.
 *
 * @author Inessa Repeshko CS-222a
 */
@WebServlet(name = "ProcessLoginServlet", value =
"/process-login")
public class ProcessLoginServlet extends HttpServlet {
    private final String LOGIN = "irepeshko";
    private final String PASSWORD = "password";
```

```

/**
 * Handles HTTP GET requests by delegating them to the {@link
#doPost} method.
 *
 * @param request the {@link HttpServletRequest} object
containing the client's request
 * @param response the {@link HttpServletResponse} object
containing the servlet's response
 * @throws IOException if an input or output error occurs
while handling the request
 */
@Override
    protected void doGet(HttpServletRequest request,
HttpServletResponse response) throws IOException {
        doPost(request, response);
    }

/**
 * Handles HTTP POST requests by validating the user's
credentials.
 *
 * <p>This method retrieves the login and password parameters
from the request and compares them to predefined values.
 * If the credentials match, the user is logged in, and the
login is stored in the servlet context.
 * The response is redirected to the welcome page.
 * If the credentials do not match, the user is redirected to
an unauthorized error page.
 *
 * @param request the {@link HttpServletRequest} object
containing the client's request
 * @param response the {@link HttpServletResponse} object
containing the servlet's response
 * @throws IOException if an input or output error occurs
while handling the request
 */

```

```

@Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response) throws IOException {
        String login = request.getParameter("login");
        String password = request.getParameter("password");

        if (login.equals(LOGIN) && password.equals(PASSWORD)) {
            getServletContext().setAttribute("login", login);
            response.setStatus(HttpServletResponse.SC_OK);
            response.sendRedirect(request.getContextPath() +
                "/hello-web-world?login=" + login);
        } else {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.sendRedirect(request.getContextPath() +
                "/unauthorized-error");
        }
    }
}

```

ДОДАТОК Р

Код класу UnauthorizedErrorServlet.java

```
package ntukhpi.csit.semit.riv.webappsrivlab1_4.servlets;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

/**
 * The UnauthorizedErrorServlet handles HTTP GET requests and
 * displays an error page
 * for unauthorized access attempts.
 *
 * <p>This servlet is mapped to the URL pattern
 * "/unauthorized-error" and is triggered when
 * a user fails to provide valid credentials or attempts to access
 * a restricted resource
 * without authorization. The servlet sets the response status to
 * 401 (Unauthorized) and
 * includes the content of the `UnauthorizedError.jsp` page to
 * display a meaningful error message.
 *
 * @author Inessa Repeshko CS-222a
 */
@WebServlet(name = "UnauthorizedErrorServlet", value =
"/unauthorized-error")
public class UnauthorizedErrorServlet extends HttpServlet {
```

```

/**
 * Handles the HTTP GET request by setting the response status
to 401 (Unauthorized) and
 * including the content of the `UnauthorizedError.jsp` page to
inform the user of the
 * unauthorized access attempt.
 *
 * @param request the {@link HttpServletRequest} object that
contains the client's request
 * @param response the {@link HttpServletResponse} object that
contains the servlet's response
 * @throws ServletException if there is an issue with the
request dispatch or JSP processing
 * @throws IOException if an input or output error occurs
during the handling of the request
 */
@Override
public void doGet(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException
{
    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);

    RequestDispatcher dispatcher =
request.getRequestDispatcher("/UnauthorizedError.jsp");
    dispatcher.include(request, response);
}
}

```


ДОДАТОК С

Код сторінки Login.jsp

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Welcome to Web World!</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<div class="container justify-content-center">
    <h1 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-3 my-5 bg-body-tertiary rounded">
        Welcome to Web World!
    </h1>

    <div class="container w-25 text-center justify-content-center">
        <h2 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
            fs-3 fw-semibold font-monospace text-center

```

```

        shadow p-2 mb-3 bg-body-tertiary rounded">
        Login
    </h2>
</div>

<div class="container w-50 text-center justify-content-center">
    <div class="card border border-2 border-info-subtle p-5
        shadow mb-5 bg-body-tertiary rounded">
        <form id="login-form" action="process-login"
method="POST">
            <div class="card-body bg-light font-monospace">
                <div class="input-group mb-5 border border-1
border-info-subtle rounded">
                    <span class="input-group-text
text-info">Login</span>
                    <input type="text" class="form-control
text-dark" name="login" required>
                </div>

                <div class="input-group mb-5 border border-1
border-info-subtle rounded">
                    <span class="input-group-text
text-info">Password</span>
                    <input type="password" class="form-control
text-dark" name="password" required>
                </div>

                <div class="d-flex justify-content-center">
                    <input type="submit" class="btn
btn-outline-info" name="submit" value="login">
                </div>
            </div>
        </form>
    </div>
</div>
</div>
</div>

```

```
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
    crossorigin="anonymous"></script>

<jsp:include page="Footer.jsp"/>
</body>
</html>
```

ДОДАТОК Т

Код сторінки UnauthorizedError.jsp

```

<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Unauthorized Error</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<div class="container justify-content-center">
    <div class="container justify-content-center w-50
position-absolute top-50 start-50 translate-middle">
        <div class="card border border-2 border-danger-subtle
            shadow mb-5 bg-danger-subtle rounded">
            <div class="card-header bg-danger-subtle border-bottom
border-2 border-danger-subtle
                font-monospace text-center
text-secondary-emphasis fw-bolder">
                <span class="badge text-bg-danger">401</span>
                Unauthorized Error
            </div>

```

```

        <div class="card-body bg-light font-monospace">
            <div class="card-text text-secondary-emphasis
fw-light text-center">
                <p>Authentication <code
class="text-danger">failed</code> due to entering invalid
credentials.</p>
                <p>Please, try again!</p>
            </div>
        </div>
    </div>

    <div class="d-flex justify-content-center mt-5">
        <a href="/WebAppsRIVLab1_4" class="btn
btn-outline-info" role="button">Go Back</a>
    </div>
</div>
</div>
</body>
</html>

```

ДОДАТОК У

Код сторінки HelloWorld.jsp (завдання 4)

```

<%@ page import="java.time.format.DateTimeFormatter" %>
<%@ page import="java.time.LocalDateTime" %>
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Hello, Web World!</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhjY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<jsp:include page="NavigationBar.jsp"/>

<div class="container justify-content-center">
    <h1 class="text-info bg-info-subtle border border-3
rounded-pill border-info-subtle
        fs-1 fw-semibold font-monospace text-center
        shadow-lg p-3 my-5 bg-body-tertiary rounded">
        <%
        StringBuilder text = new StringBuilder();
        text.append("Welcome to Web World");

```

```

        if (request.getParameter("login") != null) {
            text.append(",
        ").append(request.getParameter("login"));
        }

        text.append("!");
    %>
    <%=text.toString() %>
</h1>

<div class="row align-items-start column-gap-5">
    <div class="col w-50 p-0 card border border-2
border-info-subtle
        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">
            Student data
        </div>
        <div class="card-body bg-light font-monospace">
            <p class="card-text text-secondary-emphasis
fw-light">
                Name:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">Repeshko Inessa</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">
                Group:
                <span class="text-primary-emphasis fw-medium
fst-italic ms-3">CS-222a</span>
            </p>
            <p class="card-text text-secondary-emphasis
fw-light">

```

```

        Department:
        <span class="text-primary-emphasis fw-medium
fst-italic ms-3">SEMIT</span>
    </p>
    <p class="card-text text-secondary-emphasis
fw-light">
        Year:
        <span class="text-primary-emphasis fw-medium
fst-italic ms-3">2024</span>
    </p>
    <p class="card-text text-secondary-emphasis
fw-light">
        Page access time:
        <span class="text-primary-emphasis fw-medium
fst-italic ms-3">
            <%=
LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd.MM.yyyy
HH:mm:ss")) %>
        </span>
    </p>
</div>
</div>

<div class="col w-50 p-0 card border border-2
border-info-subtle
    shadow mb-5 bg-body-tertiary rounded">
    <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
        font-monospace text-center
text-secondary-emphasis fw-bolder">
        Employee Table
        <span class="badge text-bg-info">include()</span>
    </div>

    <div class="card-body bg-light font-monospace">

```



```

        <div class="card-text text-secondary-emphasis
fw-light">
            <p>Table of employees of some IT company.</p>
            <p>The page is displayed by including the
content.</p>
        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
            The <code class="text-info">include()</code>
method of the
            <code
class="text-info">RequestDispatcher</code> class is used to
include
            the content of another resource (servlet, JSP
page, HTML file) in the server-side response
            while preserving the data of the original
request and without changing the original URL.
        </div>

        <div class="d-flex justify-content-center">
            <a href="employee-table-by-include-method"
class="btn btn-outline-info" role="button">Show</a>
        </div>
    </div>
</div>
</div>

<div class="row align-items-start column-gap-5">
    <div class="col w-50 p-0 card border border-2
border-info-subtle
        shadow mb-5 bg-body-tertiary rounded">
        <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle
            font-monospace text-center
text-secondary-emphasis fw-bolder">

```

```

        Employee Table
        <span class="badge text-bg-info">forward()</span>
    </div>

    <div class="card-body bg-light font-monospace">
        <div class="card-text text-secondary-emphasis
fw-light">
            <p>Table of employees of some IT company.</p>
            <p>The page is displayed using a request
forwarding.</p>
        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
            The <code class="text-info">forward()</code>
method of the

            <code
class="text-info">RequestDispatcher</code> class allows forwarding
            a request from a servlet to another resource
(servlet, html-page or jsp page) on the server side
            while preserving the data of the original
request and without changing the original URL.
        </div>

        <div class="d-flex justify-content-center">
            <a href="employee-table-by-forward-method"
class="btn btn-outline-info" role="button">Show</a>
        </div>
    </div>
</div>

<div class="col w-50 p-0 card border border-2
border-info-subtle
shadow mb-5 bg-body-tertiary rounded">
    <div class="card-header bg-body-secondary border-bottom
border-2 border-info-subtle

```

```

        font-monospace text-center
text-secondary-emphasis fw-bolder">
        Employee Table
        <span class="badge
text-bg-info">sendRedirect()</span>
    </div>

    <div class="card-body bg-light font-monospace">
        <div class="card-text text-secondary-emphasis
fw-light">
            <p>Table of employees of some IT company.</p>
            <p>The page is displayed using a request
redirecting.</p>
        </div>

        <div class="border-start border-info-subtle
border-5 bg-body-secondary p-3 mb-3">
            The <code
class="text-info">sendRedirect()</code> method of the
            <code
class="text-info">HttpServletResponse</code> class is used to
redirect the client (browser)
            to another resource (URL, which can be either
an internal or external resource) on the client side
            with the loss of original data (if not passed
to the URL) and a change in the original URL.
        </div>

        <div class="d-flex justify-content-center">
            <a href="employee-table-by-sendredirect-method"
class="btn btn-outline-info" role="button">Show</a>
        </div>
    </div>
</div>
</div>
</div>

```

```
<jsp:include page="Footer.jsp"/>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
Ids1K1eN7N6jIeHz"
    crossorigin="anonymous"></script>
</body>
</html>
```

ДОДАТОК Ф

Код сторінки NavigationBar.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
    <title>Menu</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMHjY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<nav class="navbar navbar-expand-lg bg-info-subtle
text-info-emphasis font-monospace">
    <div class="container-fluid">
        <div class="mx-3">
            <a class="navbar-brand text-info " href=".">
                <svg class="bd-placeholder-img rounded me-2"
width="30" height="30" xmlns="http://www.w3.org/2000/svg"
                aria-hidden="true"
preserveAspectRatio="xMidYMid slice" focusable="false">
                    <rect width="100%" height="100%"
fill="#0dcaf0"></rect>
                </svg>
                Hello, Web World!
            </a>
        </div>

        <div class="mx-3">

```

```

        <button class="navbar-toggler" type="button"
data-bs-toggle="collapse" data-bs-target="#navbarNavAltMarkup"
        aria-controls="navbarNavAltMarkup"
aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
    </button>
</div>

<div class="collapse navbar-collapse mx-3"
id="navbarNavAltMarkup">
    <div class="navbar-nav font-monospace
text-secondary-emphasis">
        <div class="nav-item me-5">
            <a class="nav-link active" aria-current="page"
href="hello-web-world">Home</a>
        </div>

        <div class="nav-item dropdown me-5
align-content-center">
            <div class="nav-item dropdown-toggle"
data-bs-toggle="dropdown" aria-expanded="false">
                Employee Table
            </div>

            <ul class="dropdown-menu font-monospace
text-start text-secondary-emphasis">
                <li>
                    <a class="dropdown-item"
href="employee-table-by-include-method">
                        Employee Table
                        <span class="badge
text-bg-info">include()</span>
                    </a>
                </li>
                <li>

```

```

                <a class="dropdown-item"
href="employee-table-by-forward-method">
                    Employee Table
                    <span class="badge
text-bg-info">forward()</span>
                </a>
            </li>
            <li>
                <a class="dropdown-item"
href="employee-table-by-sendredirect-method">
                    Employee Table
                    <span class="badge
text-bg-info">sendRedirect()</span>
                </a>
            </li>
        </ul>
    </div>

    <div class="nav-item">
        <a class="nav-link disabled"
aria-disabled="true">Further more</a>
    </div>
</div>
</div>
</div>
</nav>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

crossorigin="anonymous"></script>
</body>
</html>

```


ДОДАТОК X

Код сторінки Footer.jsp

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
    <title>Footer</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<footer class="container-fluid mt-5 p-3 text-center fs-6
bg-info-subtle text-info-emphasis font-monospace">
    <div class="row w-100 align-items-start">
        <div class="col justify-content-around" id="footer-author">
            <div class="col mb-2 justify-content-around fw-bold
text-info">Powered by</div>
            <div id="author-name">Inessa Repeshko</div>
            <div id="author-group">CS-222a</div>
            <div id="author-company">
                <span id="company-facility">CSIT </span>
                <span id="company-department">SEMIT</span><br>
                <span id="company-name">NTU "KhPI"</span>
            </div>
        </div>

        <div class="col justify-content-around"
id="footer-address">

```

```

        <div class="col mb-2 justify-content-around fw-bold
text-info">Address</div>

        <div id="address-name">National Technical University
"Kharkiv Polytechnic Institute"</div>

        <div id="address-street">2 Kyrpychova</div>

        <div>
            <span id="address-postal-code">61002</span>
            <span id="address-locality">Kharkiv</span>,
            <span id="address-region">Ukraine</span>
        </div>
    </address>

    <address class="col justify-content-around"
id="footer-contact">
        <div class="col mb-2 justify-content-around fw-bold
text-info">Contact US</div>

        <div id="contact-telephone">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
id="contact-telephone-link"
href="tel:+38(057)7076634">+38-(057)-707-66-34</a>
        </div>

        <div id="contact-email">
            <a class="link-info link-offset-2
link-underline-opacity-25 link-underline-opacity-100-hover"
id="contact-email-link"
href="mailto:omsroot@kpi.kharkov.ua">omsroot@kpi.kharkov.ua</a>
        </div>
    </address>
</div>

<div class="row w-100 align-items-center">
    <div class="col justify-content-around"
id="footer-copyright">
        <p class="p-0 m-0">Copyright &copy; 2024 All Rights
Reserved</p>

```

```
        </div>
    </div>
</footer>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"

integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"

        crossorigin="anonymous"></script>
</body>
</html>
```

ДОДАТОК Ц

Код сторінки EmployeeTable.jsp (завдання 4)

```

<%@ page import="java.util.List" %>
<%@ page import="java.util.ArrayList" %>
<%@ page import="ntukhpi.csit.semit.riv.webappsrivlab1_4.model.*"
%>
<%@ page contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8" language="java" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <title>Employee Table</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
rap.min.css"
        rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh
0JMhY6hW+ALEwIH"
        crossorigin="anonymous">
</head>
<body>
<jsp:include page="NavigationBar.jsp"/>

<div class="mx-3 my-3 font-monospace">
    <h1 class="text-info fs-1 my-3 fw-semibold text-center">
        Employee Table
    </h1>

    <table class="table table-sm table-hover font-monospace

```

```

        table-bordered border border-3 border-info-subtle
shadow-lg">
        <thead class="table-info text-center fw-medium
text-secondary-emphasis">
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Gender</th>
            <th>Birthday</th>
            <th>Salary, UAH</th>
            <th>Program Language</th>
            <th>Importance</th>
        </tr>
        </thead>
        <tbody class="table-light fw-light text-primary-emphasis">
        <%

        String employeeIdsParam =
request.getParameter("employeeIds");

        List<Long> employeeIds = new ArrayList<>();
        List<Employee> employeesToDisplay = new ArrayList<>();

        if (employeeIdsParam != null &&
!employeeIdsParam.isEmpty()) {
            String[] ids = employeeIdsParam.split(",");

            for (String id : ids) {
                employeeIds.add(Long.parseLong(id));
            }

            EmployeeList employeeList =
EmployeeList.getInstance();

            for (Employee employee :
employeeList.getEmployees()) {
                if (employeeIds.contains(employee.getId())) {

```

```

        employeesToDisplay.add(employee);
    }
}
} else {
    employeesToDisplay = (List<Employee>)
request.getAttribute("employees");
}

    for (Employee employee : employeesToDisplay) {
        String textColor;

        switch (employee.getImportance()) {
            case 1:
                textColor = "text-warning";
                break;
            case 2:
                textColor = "text-danger";
                break;
            default:
                textColor = "text-primary";
        }

        ;

        out.println("<tr>");
        out.println("<td class=\"text-start\">" +
employee.getId() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getName() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getGenderAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getBirthDayAsString() + "</td>");
        out.println("<td class=\"text-end\">" +
employee.getSalary() + "</td>");
        out.println("<td class=\"text-start\">" +
employee.getProgramLanguage() + "</td>");

```

```

        out.println("<td class=\"text-start \" + textColor +
\"\\>\" + employee.getImportanceAsString() + "</td>");
        out.println("</tr>");
    }
    %>
</tbody>
</table>
<br>

<div class="d-flex justify-content-center">
    <a href="/WebAppsRIVLab1_4/hello-web-world" class="btn
btn-outline-info" role="button">Go Back</a>
</div>
</div>

<jsp:include page="Footer.jsp"/>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstra
p.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9Gkc
IdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
</body>
</html>

```

ДОДАТОК III

Код конфігураційного файлу web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
https://jakarta.ee/xml/ns/jakartaee/web-app_5_0.xsd"
  version="5.0">
  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>index.html</welcome-file>
    <welcome-file>Login.jsp</welcome-file>
  </welcome-file-list>
  <jsp-config>
    <jsp-property-group>
      <url-pattern>/webapp/*jsp</url-pattern>
      <el-ignored>true</el-ignored>
    </jsp-property-group>
  </jsp-config>
  <error-page>
    <error-code>401</error-code>
    <location>/UnauthorizedError.jsp</location>
  </error-page>
</web-app>
```